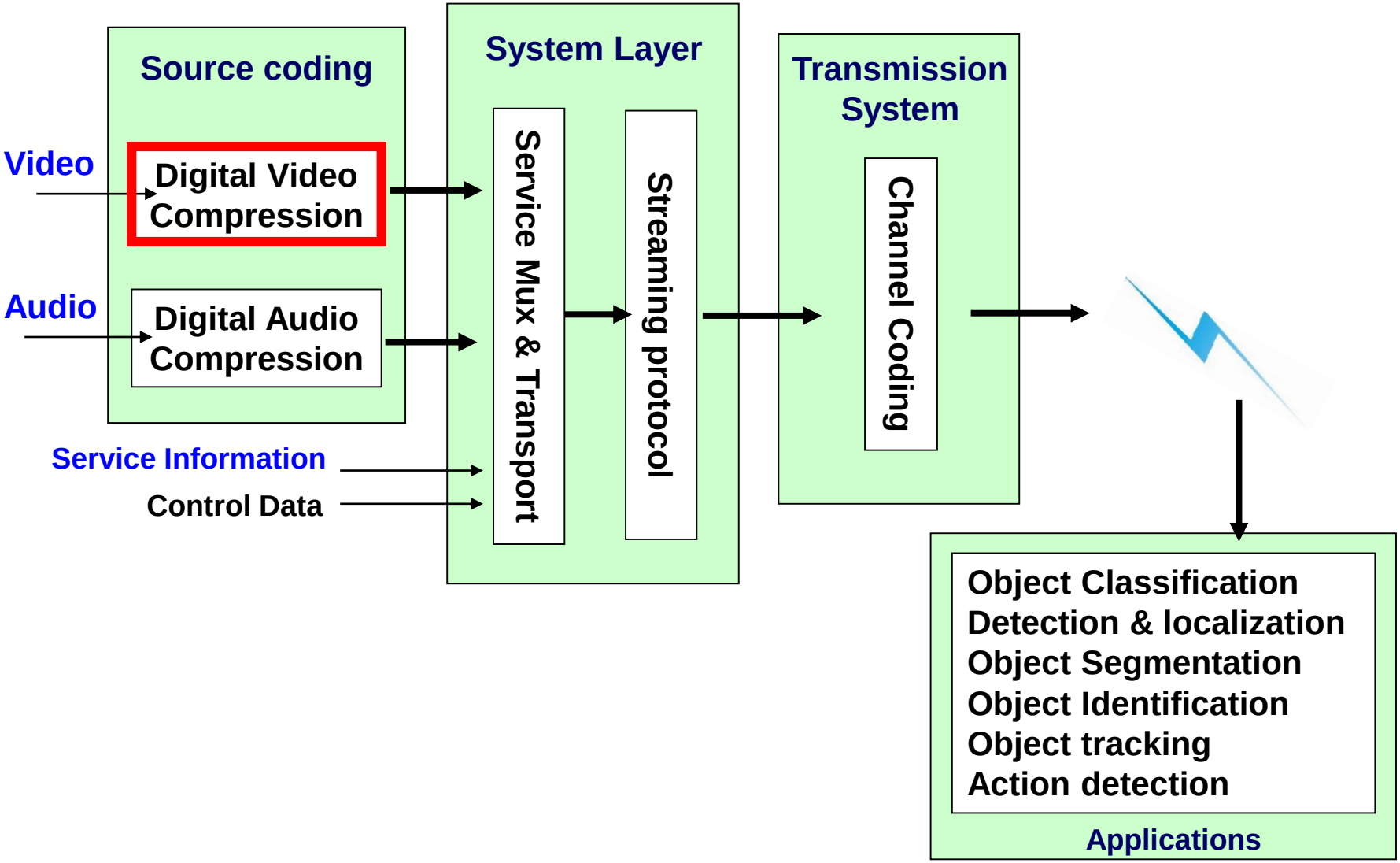
Media Streaming System

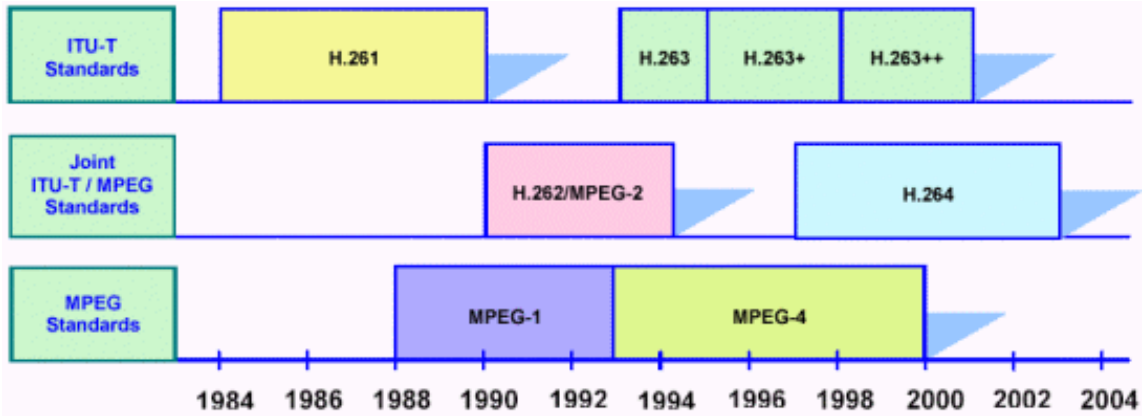


Chapter 1: Source Coding- Video

- **Video Coding – MPEG2**
  - The Basic Concept
  - Spatial Coding
  - Temporal Coding
- **Advanced Video Coding (H.264/AVC)**
- **High Efficient Video Coding (HEVC)**

Introduction

- History
  - **ITU-T** Video Coding Experts Group (VCEG): H.26x series
    - Coding of moving pictures/video for conferencing, streaming, etc.
  - **ISO** Motion Picture Experts Group (MPEG): MPEG-x series
    - Development visual parts of MPEG-1, and MPEG-4.
  - **JVT** (Joint Video Team)
    - MPEG-4 standard now includes an advanced video codec jointly developed between VCEG of ITU, and MPEG of ISO/IEC.
    - MPEG-2, H.264 (JVC), MPEG4 Part10 (AVC), HEVC, …



The Basic Concept

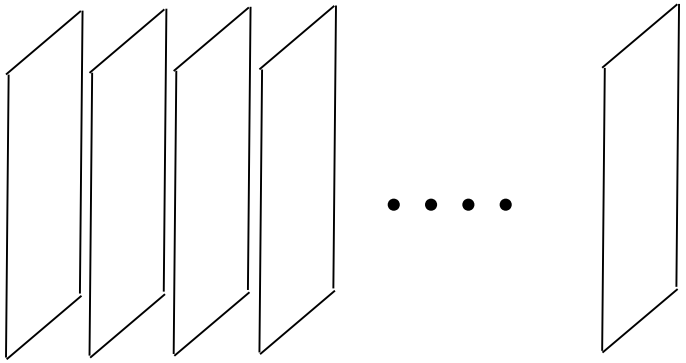
- **Compression Techniques**
  - **Lossless compression**
    - Compression methods for which the original data set can be recovered exactly from the compressed stream
    - Compression is achieved by *removing redundancy*.
  - **Lossy compression**
    - The decompression gets an imperfect reconstruction of original data.
    - Compression is achieved by
      1. *removing redundancies*
      2. *exploiting perceptual characteristics of human visual system*
- **Redundancy**
  - **Spatial redundancy**: similarities between adjacent pixels in plain areas of picture.
  - **Temporal redundancy**: similarities between successive pictures.
  - **Statistical redundancy**: same symbols occurs frequently

Video Compression

MPEG2

- **Spatial Coding (Intra-coding)**

- **Temporal Coding (Inter-coding)**

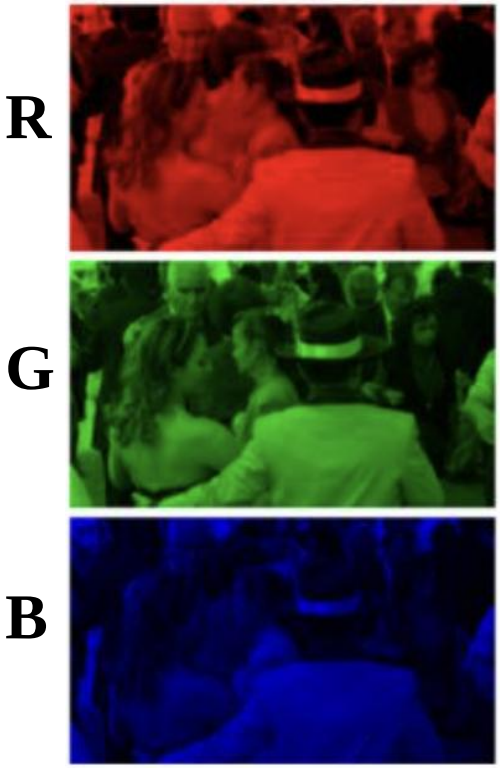


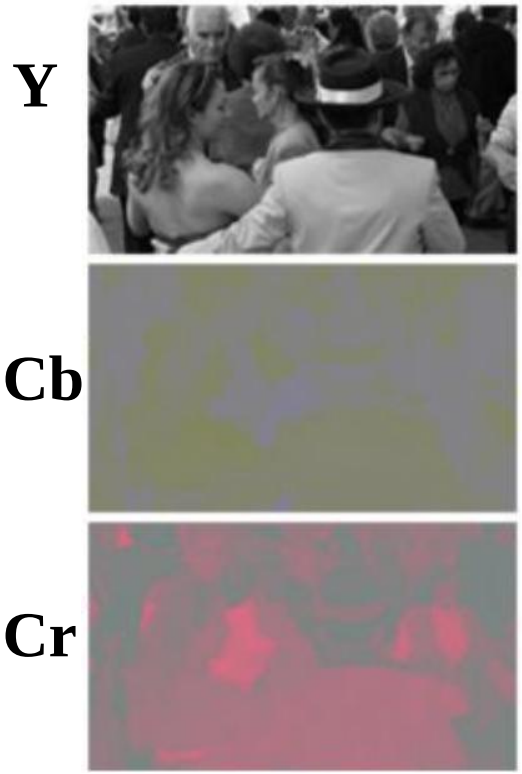
Spatial Coding

- **Spatial Coding Procedure**
  - **Color Space Conversion: RGB to YCbCr** (no loss, no data reduced)
  - **Color Sub-sampling (**information lost, data reduced**)**
  - **Discrete Cosin Transform (DCT)** (no loss, no data reduced)
  - **Quantization & Weighting** (information lost, data reduced)
  - **ZigZag Scaning** (no loss, no data reduced)
  - **Entropy Coding** (no loss, data reduced)
    - **DPCM** (Differential Pulse Code Modulation)
    - **RLC** (Run Length Coding)
    - **VLC** (Variable Length Coding)

Spatial Coding

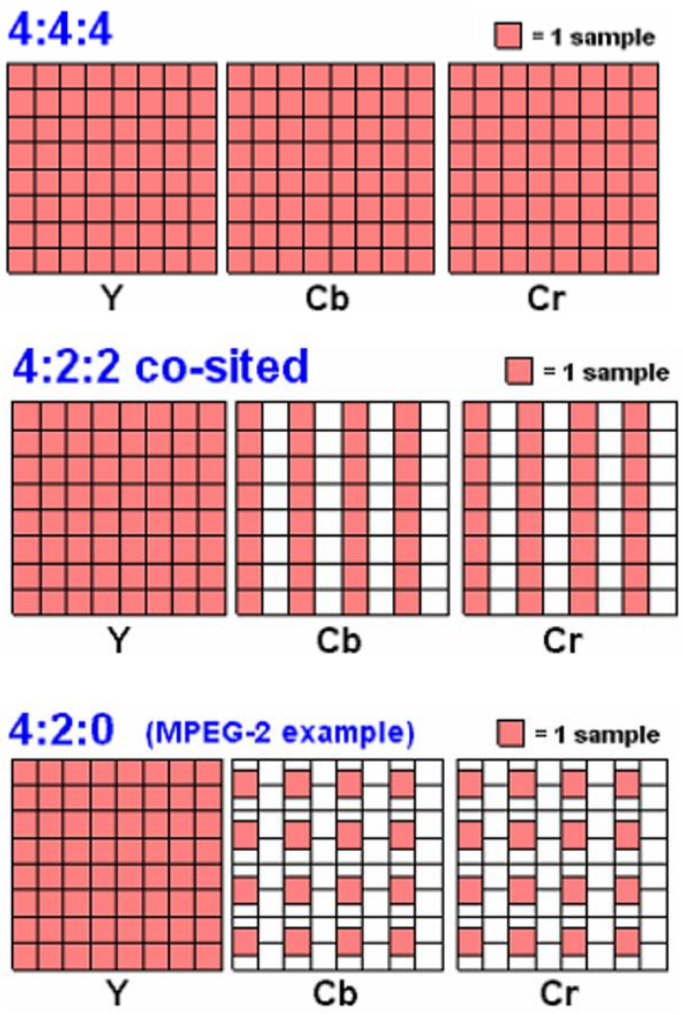
- **RGB vs. YCbCr**
  - (Y,Cb,Cr) is less correlated then (R,G,B) , so we can encode the (Y,Cb,Cr) components more efficiently.





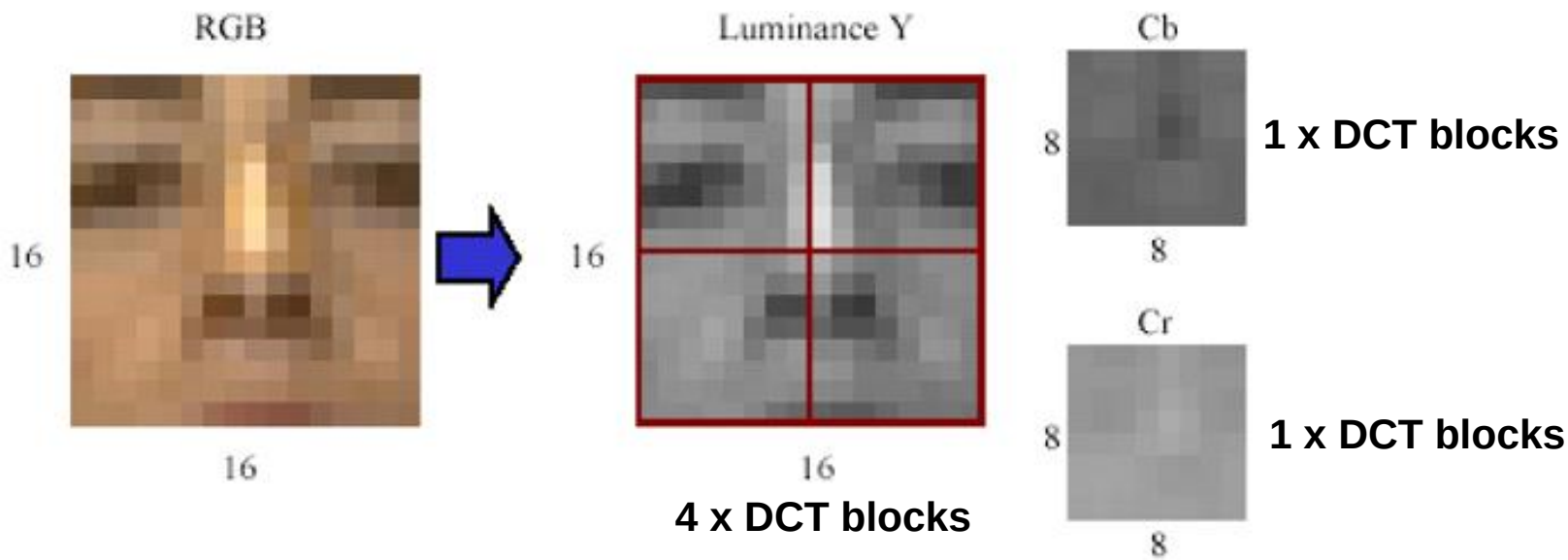
Spatial Coding

- **Chroma Sub-sampling on YCbCr**
  - Use low resolution for chrominance because eye is more sensitive to grayscale than to color.
  - Chroma (Cb and Cr) are sampled at a lower rate than Luma (Y).
  - **4:4:4** – Cb and Cr are sampled at the same rate as Y
  - **4:2:2** – Cb and Cr are sampled at 1/2 resolution of Y. (DV, Digital Betacam)
  - **4:2:0** – Cb and Cr are sampled at 1/4 resolution of Y. (MPEG-1,2)



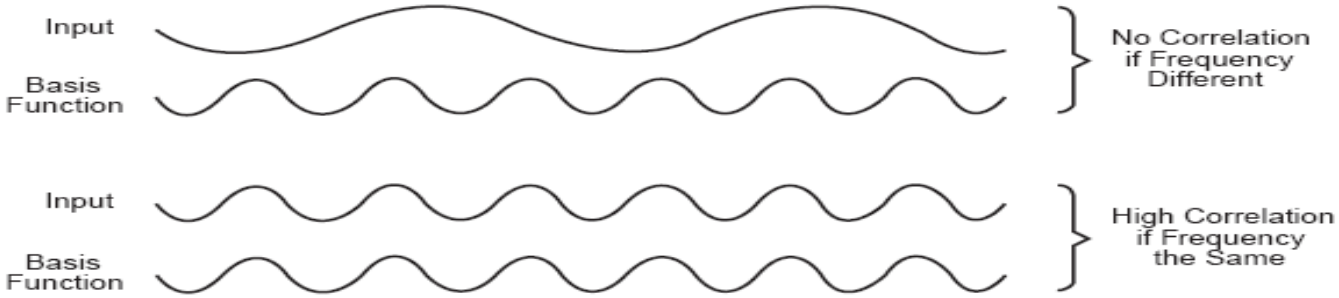
Spatial Coding

- **Color Space Conversion**
  - **RGB → YCbCr**
- **Color Sub-sampling**
  - **4:2:0:** Cb and Cr are sampled at one quarter resolution of Y



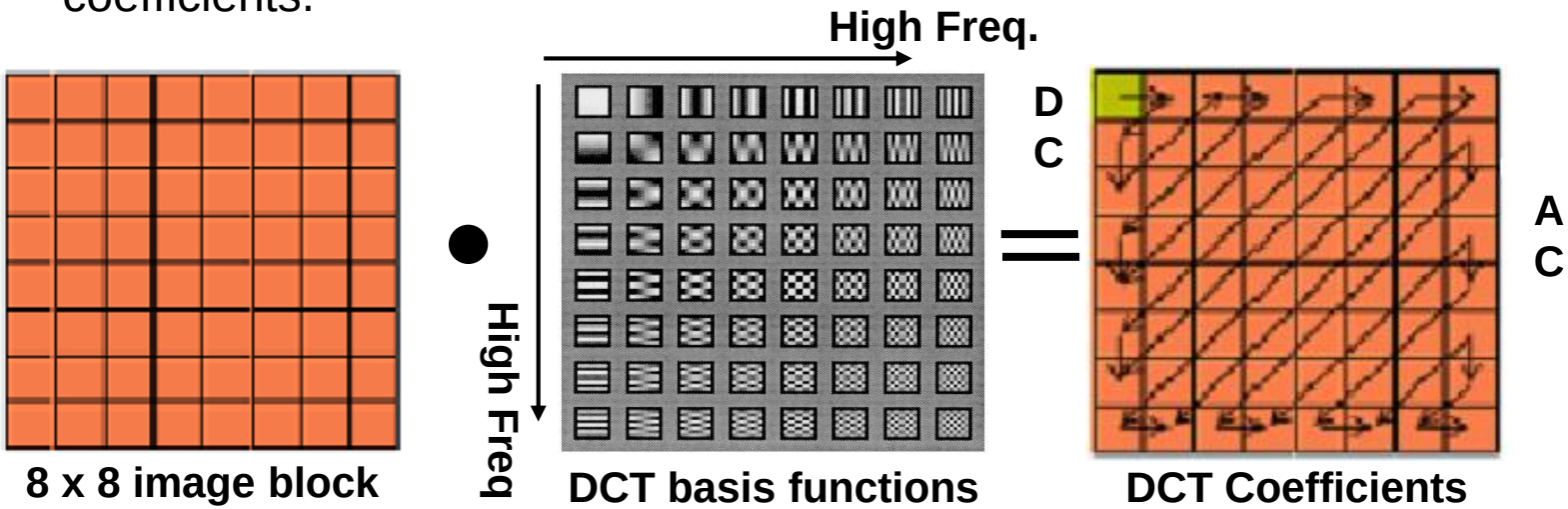
Spatial Coding

- **Transform**
  - A transform is a way of expressing waveform in a different domain, in MPEG case, the frequency domain.
  - The output of a transform is a set of coefficients that describe how a given frequency is present.
  - **Fourier transform**
    - Fourier transform: it finds each frequency in the input signal by multiplying the input waveform by a target freq (called a basis function), and integrating the product.
    - If the input waveform does not contain the target freq, the integral will be zero; otherwise, the integral will be a coeff describing the amplitude of that component freq.



Spatial Coding

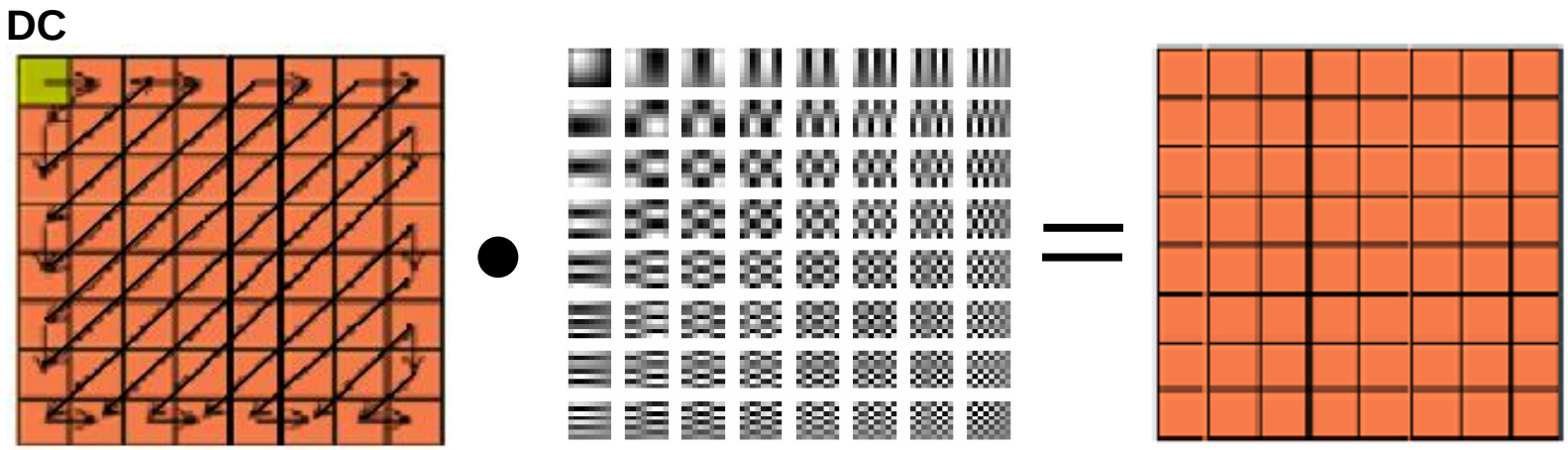
- **DCT (Discrete cosine transform)**
  - The DCT is the sampled version of the cosine transform,
  - Use 64 basis functions in two-dimensional form in MPEG.
  - **DCT block**: In 4:2:0 video, each 16x16 partition includes 6 DCT blocks (4Y + 1Cr + 1Cb). A block of 8x8 pixels is transformed to be a block of 8x8 coefficients.



$$F(u,v) = \frac{C(u)C(v)}{4} \sum_{i=0}^{7} \sum_{j=0}^{7} \cos\frac{(2i+1)u\pi}{16} \cos\frac{(2j+1)v\pi}{16} f(i,j) \qquad C(\xi) = \begin{cases} \frac{\sqrt{2}}{2} & \text{if } \xi = 0 \\ 1 & \text{otherwise} \end{cases}$$

Spatial Coding

- **IDCT (Inverse Discrete Cosine Transform)**



DCT Coefficients
$F(u,v)$

IDCT basis functions

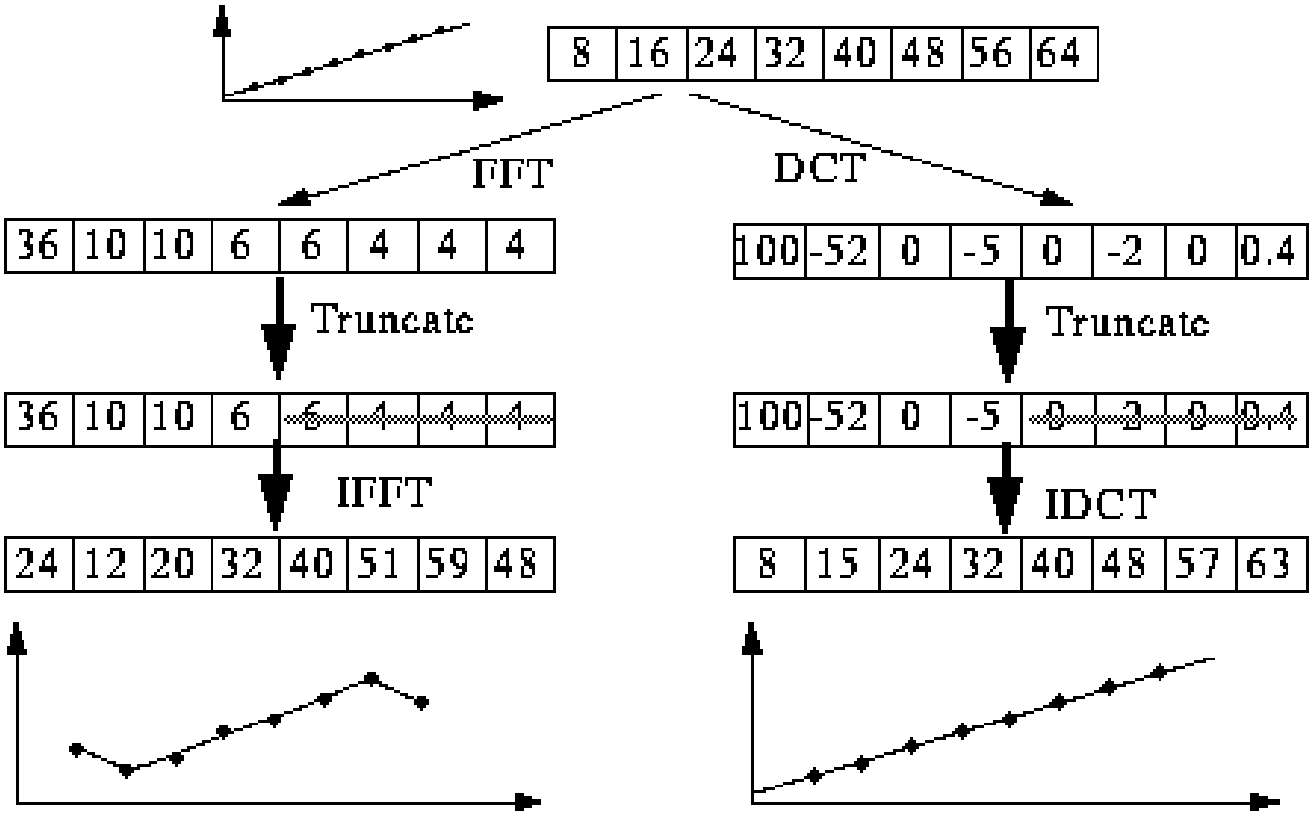
8 x 8 image block
$f(i,j)$

$$\tilde{f}(i,j) = \sum_{u=0}^{7}\sum_{v=0}^{7} \frac{C(u)C(v)}{4} \cos\frac{(2i+1)u\pi}{16} \cos\frac{(2j+1)v\pi}{16} F(u,v)$$

$$C(\xi) = \begin{cases} \frac{\sqrt{2}}{2} & \text{if } \xi = 0 \\ 1 & \text{otherwise} \end{cases}$$

Spatial Coding

- **Why DCT not FFT (Fast Fourier Transform) ?**
  - **Energy compaction**



Spatial Coding

- **DCT energy compaction:**
  - DCT Transforms each block of 8x8 samples into a block of 8x8 spatial frequency coeff (-1024 ~1023). Generally, energy tends to be concentrated into a few significant coeff; other coeffs are close to zero.

52	55	61	66	70	61	64	73
63	59	66	90	109	85	69	72
62	59	68	113	144	104	66	73
63	58	71	122	154	106	70	69
67	61	68	104	126	88	68	70
79	65	60	70	77	68	58	75
85	71	64	59	55	61	65	83
87	79	69	68	65	76	78	94

Pixel domain

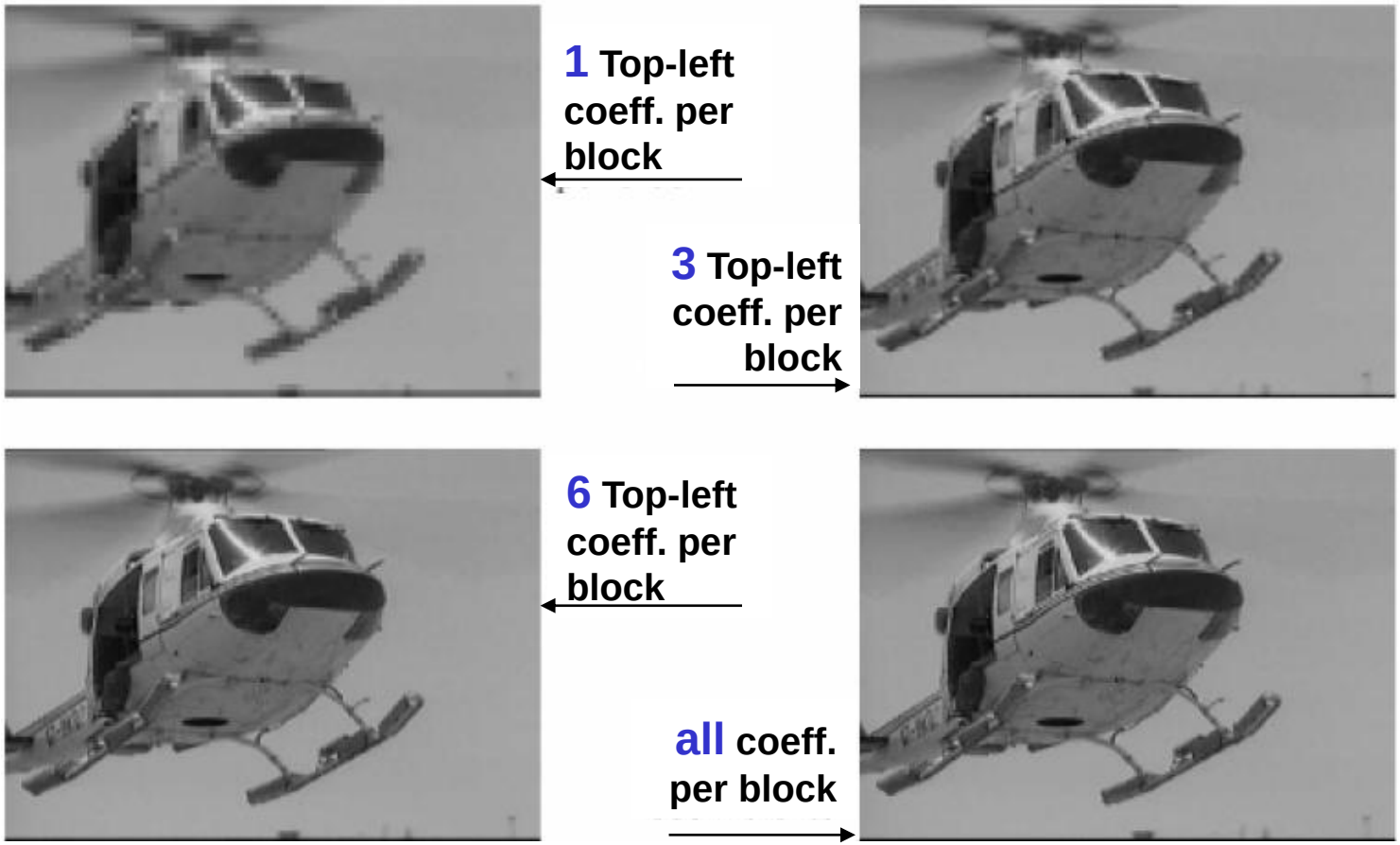
DCT ↓ ↑ IDCT

609	−29	−62	25	55	−20	−1	3
7	−21	−62	9	11	−7	−6	6
−46	8	77	−25	−30	10	7	−5
−50	13	35	−15	−9	6	0	3
11	−8	−13	−2	−1	1	−4	1
−10	1	3	−3	−1	0	2	−1
−4	−1	2	−1	2	−3	1	−2
−1	−1	−1	−2	−1	−1	0	−1

Frequency domain

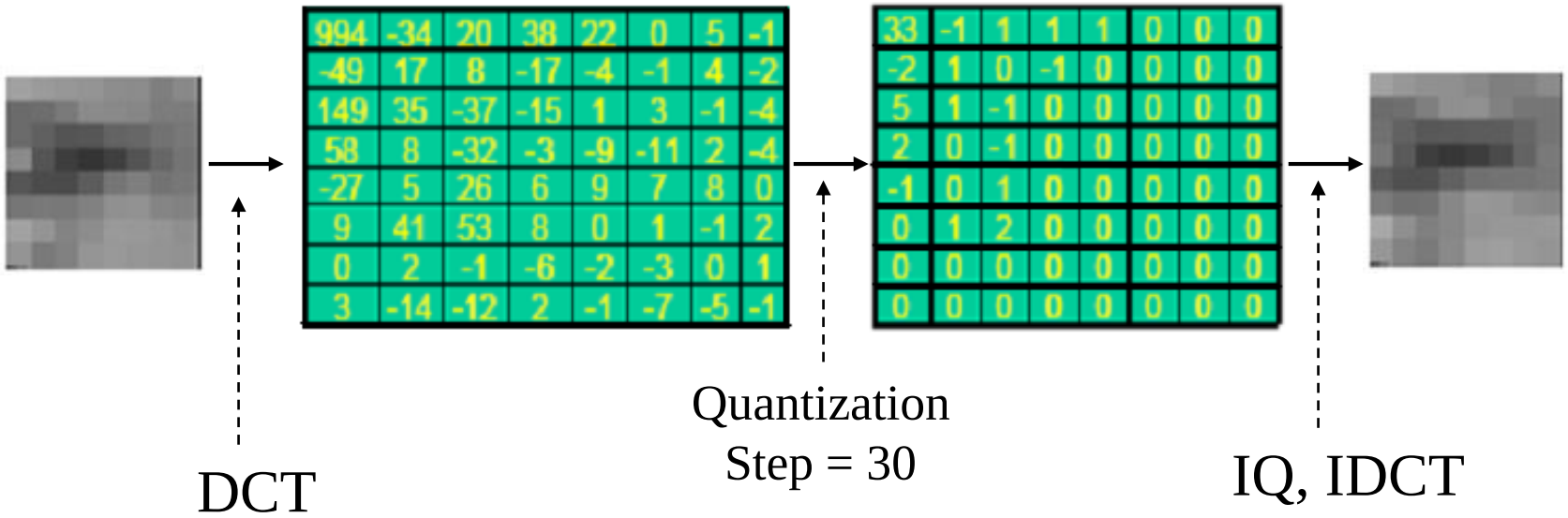
Spatial Coding

- **DCT energy compaction**
  - Most image blocks only contain a few significant coeff. (usually the “lowest frequency coeff”)



Spatial Coding

- **Quantization**
  - Represent coef. using less steps (fewer bits) to obtain compression.
  - Divide each DCT coefficient by an integer, discard remainder
  - Result: loss of precision
  - Typically, a few non-zero coefficients are left



Spatial Coding

- **Weighted Quantization**
  - human perception of noise in pictures is not uniform but is a function of the spatial frequency. More noise can be tolerated at high spatial frequencies
  - With weighted quantization, low-frequency coefficients can be divided by small numbers (small steps), while high-frequency coefficients by large numbers (large step).
  - A matrix called quantizer (Q[u,v]) is used to define quantization step for DCT coefficient matrix (F[u,v]).
    - **Quantized DCT coefficient** :

$$\hat{F}(u,v) = round\left(\frac{F(u,v)}{Q(u,v)}\right)$$

7842	199	448	362	342	112	31	22
198	151	181	264	59	37	14	3
142	291	218	87	27	88	27	12
111	133	159	119	58	65	36	2
49	85	217	50	8	3	14	12
58	120	60	40	41	11	2	1
30	121	61	22	30	1	0	1
22	28	2	33	24	51	44	81

Input DCT Coefficients
(a more complex block)

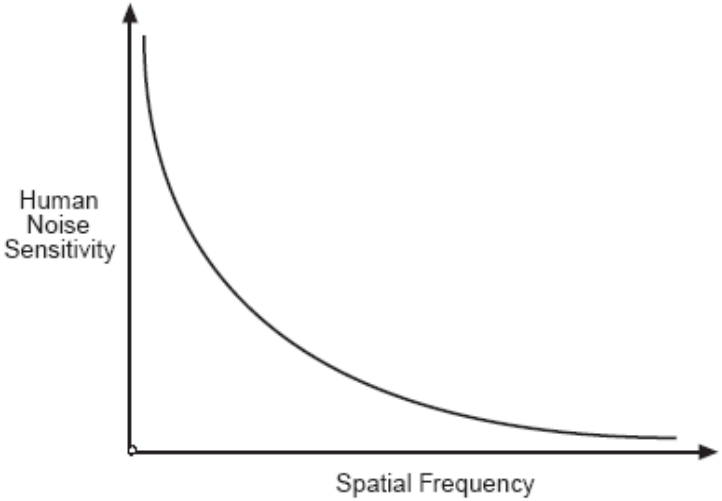
Divide by Quant Matrix

980	12	23	16	13	4	1	0
12	9	8	11	2	1	0	0
7	13	8	3	0	2	0	1
5	6	6	4	2	1	0	0
2	3	8	1	0	0	0	0
2	4	2	1	1	0	0	0
1	4	2	1	0	0	0	0
0	0	1	0	0	0	0	0

Output DCT Coefficients
Value for display only
not actual results

8	16	19	22	26	27	29	34
16	16	22	24	27	29	34	37
19	22	26	27	29	34	34	38
22	22	26	27	29	34	37	40
22	26	27	29	32	35	40	48
26	27	29	32	35	40	48	58
26	27	29	34	38	48	56	69
27	29	35	38	46	56	69	83

Quant Matrix Values
Value used corresponds
to the coefficient location



Example 1: DCT, Quantization, Dequantization, IDCT

Smooth block

200	202	189	188	189	175	175	175
200	203	198	188	189	182	178	175
203	200	200	195	200	187	185	175
200	200	200	200	197	187	187	187
200	205	200	200	195	188	187	175
200	200	200	200	200	190	187	175
205	200	199	200	191	187	187	175
210	200	200	200	188	185	187	186

$f(i,j)$

DCT

515	65	-12	4	1	2	-8	5
-16	3	2	0	0	-11	-2	3
-12	6	11	-1	3	0	1	-2
-8	3	-4	2	-2	-3	-5	-2
0	-2	7	-5	4	0	-1	-4
0	-3	-1	0	4	1	-1	0
3	-2	-3	3	3	-1	-1	3
-2	5	-2	4	-2	2	-3	0

$F(u,v)$

1	6	-2	2	7	-3	-2	-1
-1	4	2	-4	1	-1	-2	-3
0	-3	-2	-5	5	-2	2	-5
-2	-3	-4	-3	-1	-4	4	8
0	4	-2	-1	-1	-1	5	-2
0	0	1	3	8	4	6	-2
1	-2	0	5	1	1	4	-6
3	-4	0	6	-2	-2	2	2

Error

$\epsilon(i,j) = f(i,j) - \tilde{f}(i,j)$

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

$Q(u,v)$

$$\hat{F}(u,v) = round\left(\frac{F(u,v)}{Q(u,v)}\right)$$

Quantization

32	6	-1	0	0	0	0	0
-1	0	0	0	0	0	0	0
-1	0	1	0	0	0	0	0
-1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

$\hat{F}(u,v)$

$$\tilde{F}(u,v) = \hat{F}(u,v)\,Q(u,v)$$

Dequantization

512	66	-10	0	0	0	0	0
-12	0	0	0	0	0	0	0
-14	0	16	0	0	0	0	0
-14	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

$\tilde{F}(u,v)$

IDCT

199	196	191	186	182	178	177	176
201	199	196	192	188	183	180	178
203	203	202	200	195	189	183	180
202	203	204	203	198	191	183	179
200	201	202	201	196	189	182	177
200	200	199	197	192	186	181	177
204	202	199	195	190	186	183	181
207	204	200	194	190	187	185	184

$\tilde{f}(i,j)$

Example 2: DCT, Quantization, Dequantization, IDCT

Textured block

70	70	100	70	87	87	150	187
85	100	96	79	87	154	87	113
100	85	116	79	70	87	86	196
136	69	87	200	79	71	117	96
161	70	87	200	103	71	96	113
161	123	147	133	113	113	85	161
146	147	175	100	103	103	163	187
156	146	189	70	113	161	163	197

$f(i,j)$

DCT

-80	-40	89	-73	44	32	53	-3
-135	-59	-26	6	14	-3	-13	-28
47	-76	66	-3	-108	-78	33	59
-2	10	-18	0	33	11	-21	1
-1	-9	-22	8	32	65	-36	-1
5	-20	28	-46	3	24	-30	24
6	-20	37	-28	12	-35	33	17
-5	-23	33	-30	17	-5	-4	20

$F(u,v)$

Error

0	10	-6	-24	25	-16	4	11
0	-1	11	4	-15	27	-6	-31
2	-14	24	-23	-4	-11	-3	29
4	16	-24	20	24	1	11	-49
-12	13	-27	-7	-8	-18	12	23
-3	0	16	-2	-20	21	0	-1
5	-12	6	27	-3	2	14	-37
6	5	-6	-9	6	14	-47	44

$\epsilon(i,j) = f(i,j) - \tilde{f}(i,j)$

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

$Q(u,v)$

$$\hat{F}(u,v) = round\left(\frac{F(u,v)}{Q(u,v)}\right)$$

Quantization

-5	-4	9	-5	2	1	1	0
-11	-5	-2	0	1	0	0	-1
3	-6	4	0	-3	-1	0	1
0	1	-1	0	1	0	0	0
0	0	-1	0	0	1	0	0
0	-1	1	-1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

$\hat{F}(u,v)$

$$\tilde{F}(u,v) = \hat{F}(u,v)\,Q(u,v)$$

Dequantization

-80	-44	90	-80	48	40	51	0
-132	-60	-28	0	26	0	0	-55
42	-78	64	0	-120	-57	0	56
0	17	-22	0	51	0	0	0
0	0	-37	0	0	109	0	0
0	-35	55	-64	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

$\tilde{F}(u,v)$

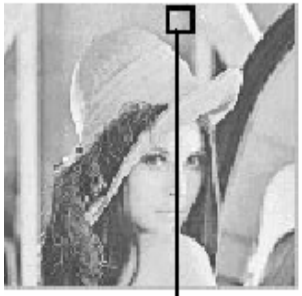
IDCT

70	60	106	94	62	103	146	176
85	101	85	75	102	127	93	144
98	99	92	102	74	98	89	167
132	53	111	180	55	70	106	145
173	57	114	207	111	89	84	90
164	123	131	135	133	92	85	162
141	159	169	73	106	101	149	224
150	141	195	79	107	147	210	153

$\tilde{f}(i,j)$

Comparison of two examples

The error is larger for the textured block so more loss is introduced.



DCT → Quantization → Dequantization → IDCT → (−) (+)

Smooth block

200	202	189	188	189	175	175	175
200	203	198	188	189	182	178	175
203	200	200	195	200	187	185	175
200	200	200	200	197	187	187	187
200	205	200	200	195	188	187	175
200	200	200	200	200	190	187	175
205	200	199	200	191	187	187	175
210	200	200	200	188	185	187	186

$f(i,j)$

Error

1	6	-2	2	7	-3	-2	-1
-1	4	2	-4	1	-1	-2	-3
0	-3	-2	-5	5	-2	2	-5
-2	-3	-4	-3	-1	-4	4	8
0	4	-2	-1	-1	-1	5	-2
0	0	1	3	8	4	6	-2
1	-2	0	5	1	1	4	-6
3	-4	0	6	-2	-2	2	2

$\epsilon(i,j) = f(i,j) - \tilde{f}(i,j)$

Textured block

70	70	100	70	87	87	150	187
85	100	96	79	87	154	87	113
100	85	116	79	70	87	86	196
136	69	87	200	79	71	117	96
161	70	87	200	103	71	96	113
161	123	147	133	113	113	85	161
146	147	175	100	103	103	163	187
156	146	189	70	113	161	163	197

$f(i,j)$

Error

0	10	-6	-24	25	-16	4	11
0	-1	11	4	-15	27	-6	-31
2	-14	24	-23	-4	-11	-3	29
4	16	-24	20	24	1	11	-49
-12	13	-27	-7	-8	-18	12	23
-3	0	16	-2	-20	21	0	-1
5	-12	6	27	-3	2	14	-37
6	5	-6	-9	6	14	-47	44

$\epsilon(i,j) = f(i,j) - \tilde{f}(i,j)$

DCT → Quantization → Dequantization → IDCT → (+) (−)

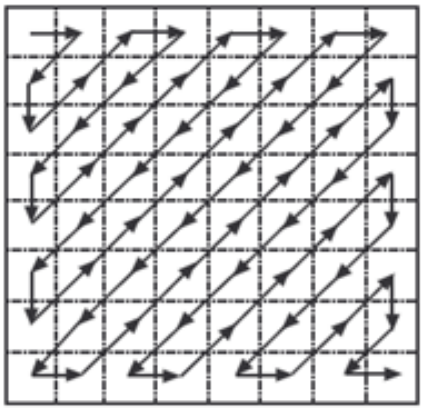
Spatial Coding

- **Zig-zag Scanning**
  - After weighted quantization, low-value coef. might be truncated to zero. More efficient transmission can be obtained if all the non-zero coef. are sent first, followed by a code indicating that the remainder are all zero. Zig-zag scanning increases the probability to achieve this.

Quantized DCT coefficients

32	6	-1	0	0	0	0	0
-1	0	0	0	0	0	0	0
-1	0	1	0	0	0	0	0
-1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Zigzag



32 6 -1 -1 0 -1 0 0 0 -1 0 0 1 0 0 0 • • •

Spatial Coding

- **Entropy Coding**
  - Despite the use of zig-zag scanning, zero coef. will still appear between the significant values. DPCM, RLC and VLC allows these coef. to be handled more efficiently.

  – **RLC** (Run Length Coding), **RLE**(Run Length Encoding)
    - Where repeating values (say x), are present, RLC simply transmits the number of x, rather than each individual bit.
    - **AC coeff.** ,may consist of a lots of zero.

  – **DPCM** (Differential Pulse Code Modulation)
    - **DC coeff.** is unlikely to change drastically within a short distance so differential coding is expected to have small magnitude.

  – **VLC** (Variable Length Coding)
    - to achieve further compression by converting frequently occurring values to short code , and infrequent values to long code.

Spatial Coding

- **RLC(Run Length Coding)**
  - Used to code AC coefficients for each block
  - ***(RunLength,Value)*** : *RunLength* is the number of zeros to skip, *Value* is the next non-zero value.
  - ***(RunLength, Size, Amplitude)*** : *Size* is the number of bits needed for representing Amplitude, and the *Amplitude* is the actual bits.

Quantized DCT coefficients

32	6	-1	0	0	0	0	0
-1	0	0	0	0	0	0	0
-1	0	1	0	0	0	0	0
-1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

DCT coefficients → 32 6 -1 -1 0 -1 0 0 0 -1 0 0 1 0 0 0 · · ·

DC coefficient

RLC

EOB

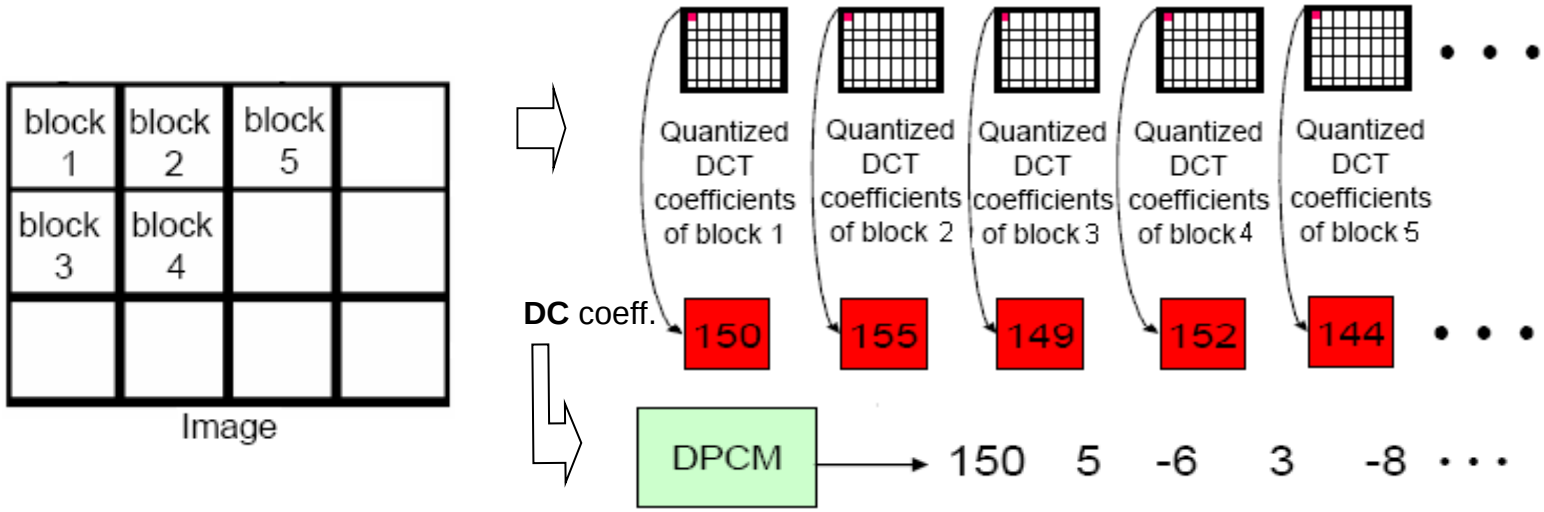
(0,6) (0,-1) (0,-1) (1,-1) (3,-1) (2,1) (0,0)

(RunLength, Size, Amplitude) : (0,3, 110) (0,1, 0) (0,1, 0) (1,1, 0) (3,1, 0) (2,1, 1) (0, 0)

Huffman coded **Huffman coded**

Spatial Coding

- **DPCM (Differential Pulse Code Modulation)**
  - Used to encode DC coefficients for each block
  - **Slice:** a groups of MBs for DPCM to apply



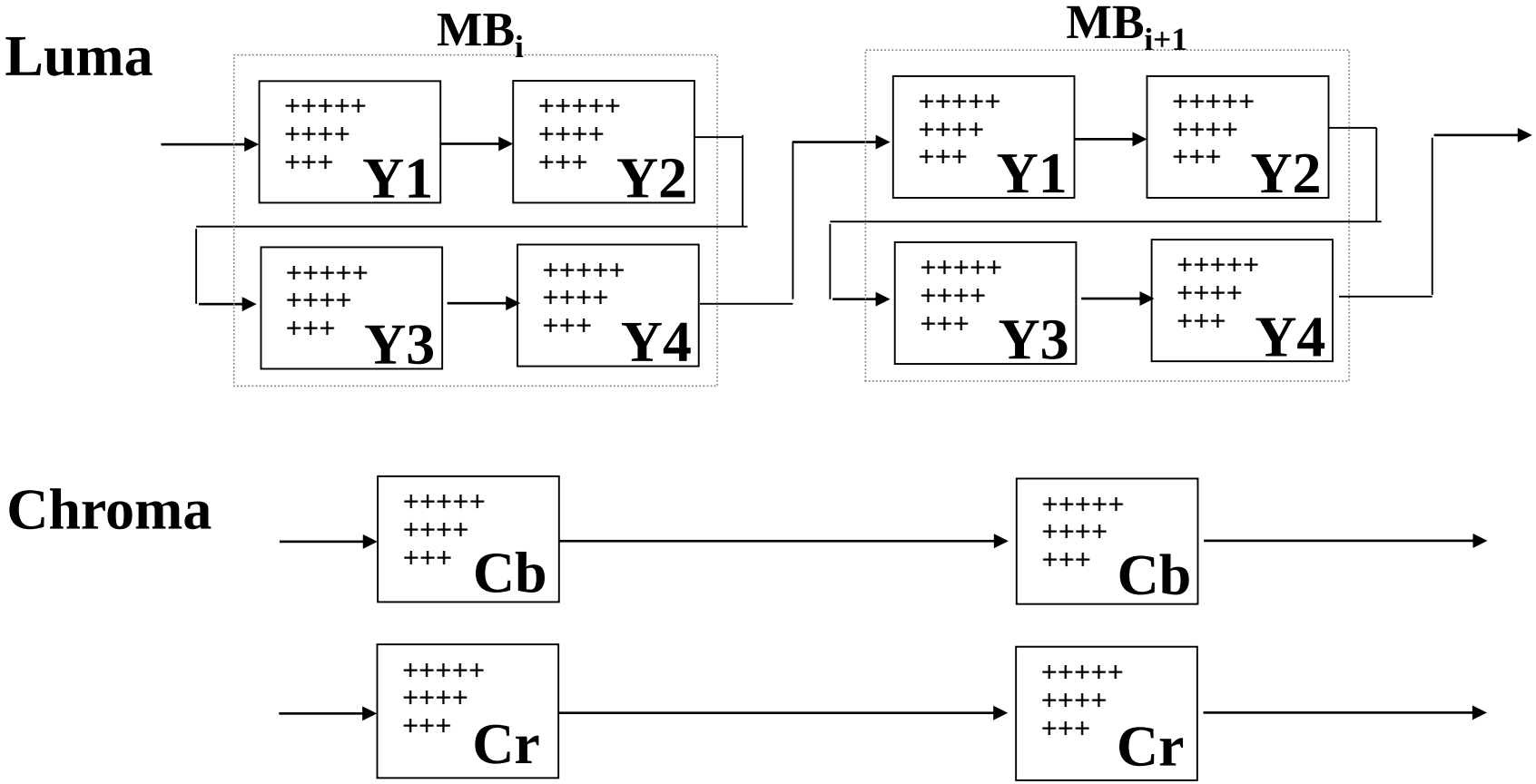
$150_{(10)} = 10010110_{(2)}$ $5_{(10)} = 101_{(2)}$ $6_{(10)} = 110_{(2)}$ $3_{(10)} = 11_{(2)}$ $-8_{(10)} = 1000_{(2)}$

***(Size, Amplitude)*: (8,10010110) (3,101) (3,001) (2,11) (4,0111)**

Huffman coded

Spatial Coding

- **DPCM (Differential Pulse Code Modulation)**
  - DC Prediction Sequence (1MB = 4Y+1Cb+1Cr)



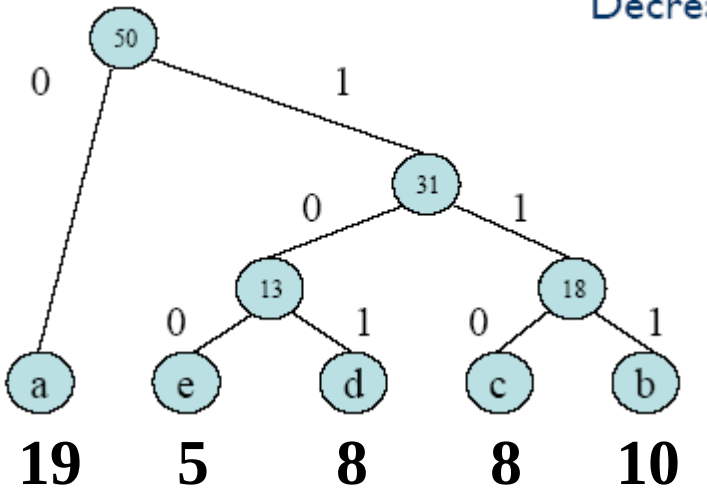
Spatial Coding

- **VLC (Variable Length Coding)**
  - Huffman Coding

1. Show symbols as leave nodes

Symbol	a	b	c	d	e
Frequency	19	10	8	8	5

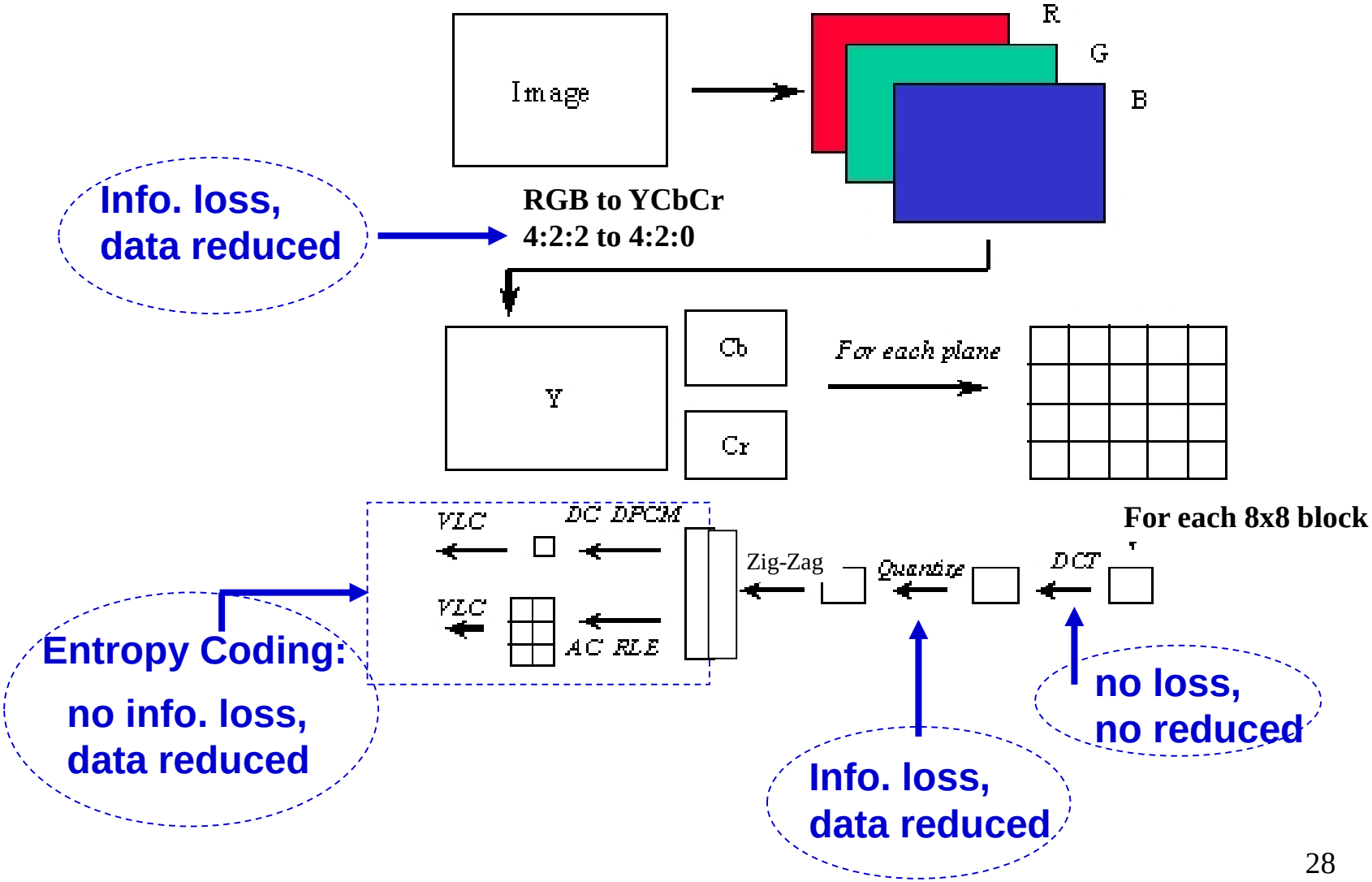
2. Combine nodes with lowest frequencies…



3. Fewer bits for symbols closer to root

Symbol	Code
a	0
b	111
c	110
d	101
e	100

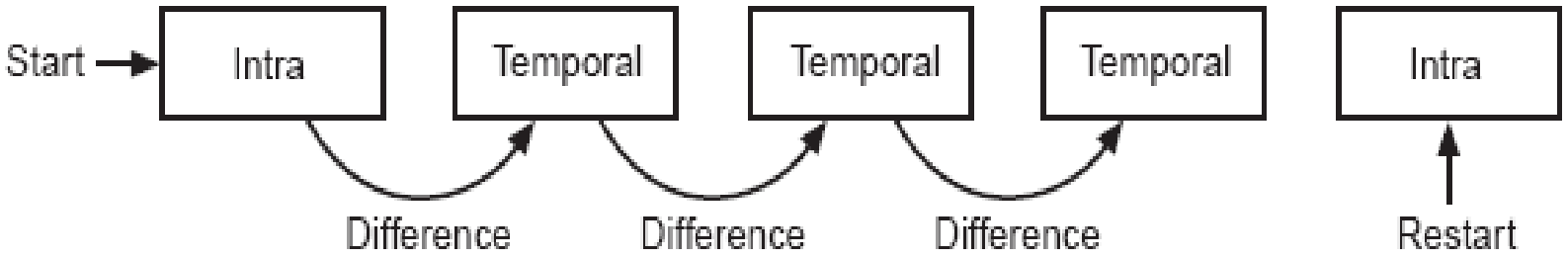
A Spatial Encoder



Temporal Coding

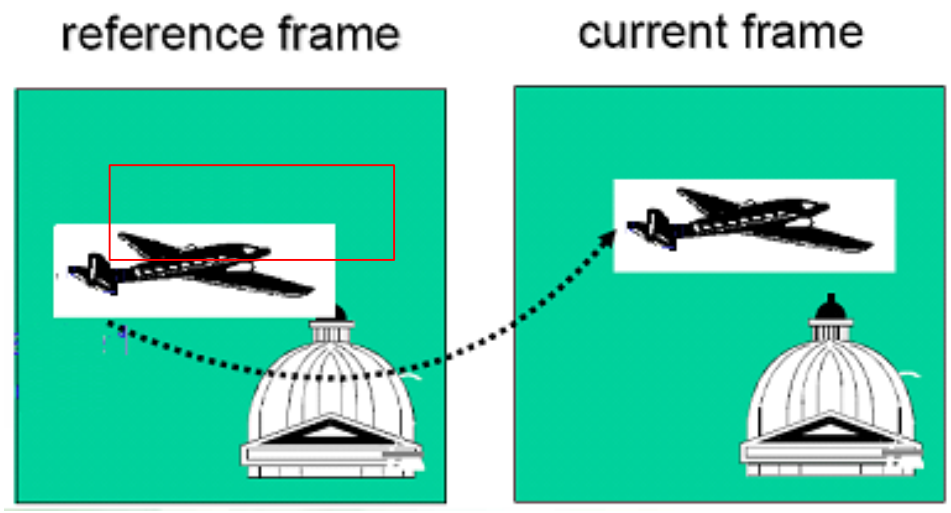
Temporary Coding

- **Temporary Coding**
  - Taking advantage of the similarity between pictures. Temporal redundancy can be reduced by transmitting only the differences between pictures.
  - Periodically send complete pictures, which are called *Intra-coded frames* (or I-frames), and are obtained by spatial compression only.
    - Stop error propagate indefinitely,
    - To allow decoding from intermediate frames of the sequence (no need to decode from the beginning of the sequence)



Temporary Coding

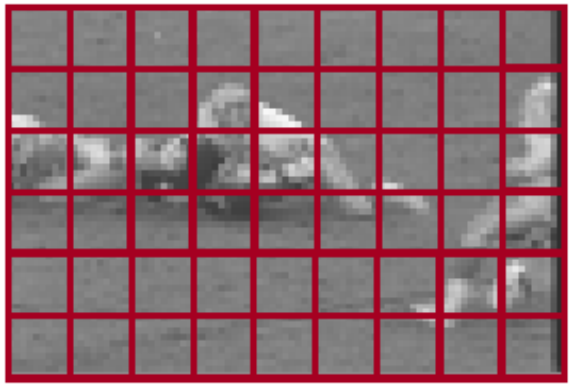
- **Motion Compensation**
  - Motion reduces the similarities between pictures and increases the data needed to create the difference picture. Motion compensation is used to increase the similarity.
  - when an object moves across TV screen, it may appear in different places in each picture, but does not change its appearance a lot. The picture difference can be reduced by measuring the motion at the encoder, and sent to the decoder as a vector, called **motion vector**.



Temporary Coding

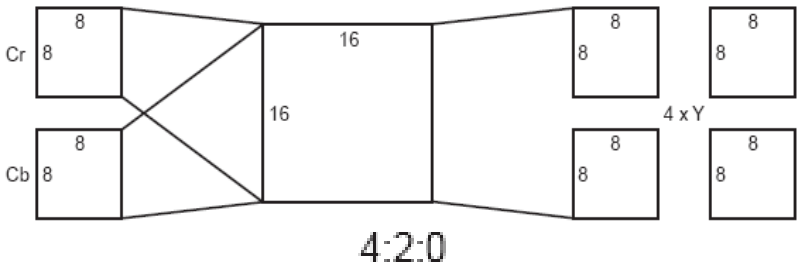
- **Motion Compensation**
  - **Ideally –** motion for each pixel
    - Too expensive
  - **Semantically** – motion for each homogeneous region or object
    - MPEG-4 Part 2, but too complex
  - **Simplified** – block-matching motion estimation (widely used)
    - Motion for each 16x16 block called **macroblock (MB)**
    - Search for an identical or similar block in the reference image

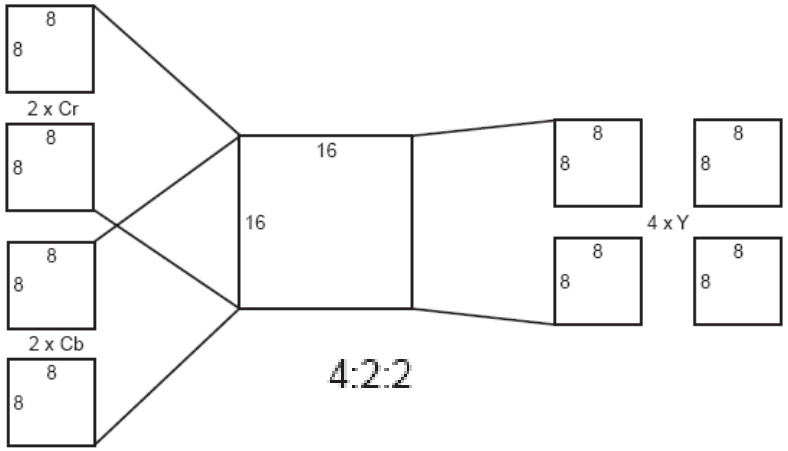
    - *DCT block is 8x8.*
      Why MB uses 16x16?



Temporary Coding

- **Macroblock (MB)**
  - MB is the minimum area to be shifted by a motion vector.
  - The MB size is determined by the DCT coding and the color sub-sampling structure.
  - ➢ 4:2:0 – a MB contains four Y blocks, one Cr and one Cb block. (4Y, 1Cr, 1Cb)
  - ➢ 4:2:2 – a MB contains four Y blocks, two Cr and two Cb blocks. (4Y, 2Cr, 2Cb)



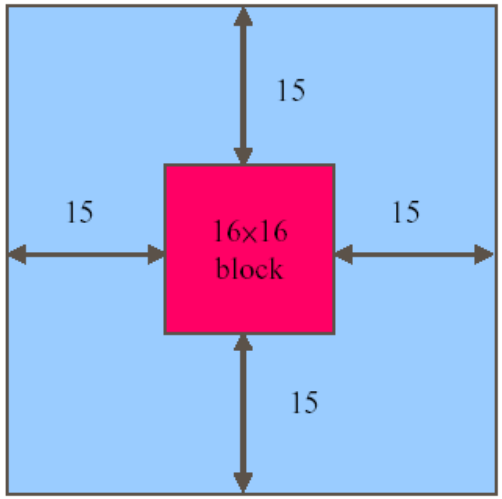


Temporary Coding

- **Motion Model**
  - **Affine model** – translation, rotation, scaling
    - 6 parameters in 2-D space
    - Complex
  - **Simplified translation model** (used by standard)
    - For each MB in the current frame, all possible displacement in the reference frame is measured.
    - **Motion vectors** (2 parameters)
    - **Coding of motion vectors**
      - A moving object may consist a number of MBs. A greater compression factor is obtained if the MVs are transmitted *differentially.* Since adjacent MBs may have the same vectors, the vector difference becomes zero.

Temporary Coding

- **Block-matching Motion Estimation**
  - Investigate possible motions within a **search window**
  - Look for one with minimum MSE or SAD



- **Matching Criteria**
  - **MSE** (Mean Squared Error)

$$\mathrm{MSE}_{(u,v)} = \sum_{i=0}^{15}\sum_{j=0}^{15}[f_t(i,j) - f_{t-1}(i+u, j+v)]^2, \ -15 \le u, v \le 15$$

  - **SAD** (Sum of Absolute Difference)

$$\mathrm{SAD}_{(u,v)} = \sum_{i=0}^{15}\sum_{j=0}^{15}|f_t(i,j) - f_{t-1}(i+u, j+v)|, \ -15 \le u, v \le 15$$

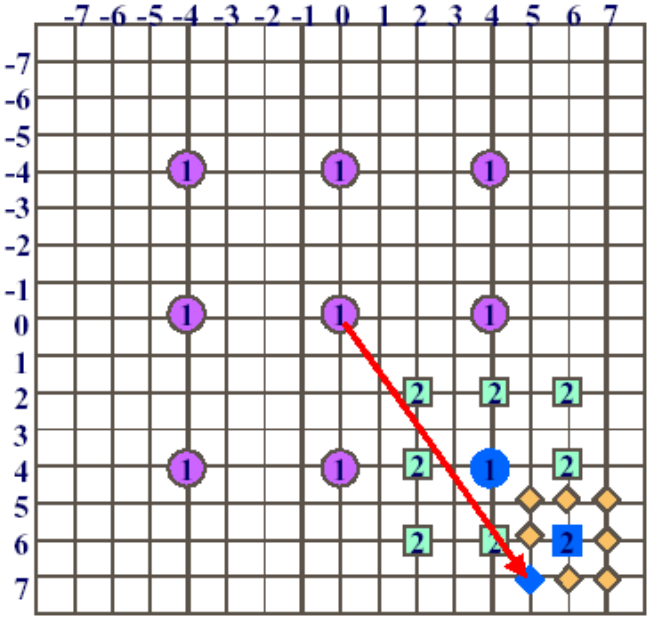
Temporary Coding

- **Motion Estimation Algorithms** (not defined by standard)
  - **Full Search**
    - Guarantee to find the global minimum SAD
    - high computational complexity
  - **Fast Search**
    - Local minimum SAD
    - Low computational complexity
    - Reduce candidate blocks
  - Compromise between efficiently removing temporal redundancy and computational requirement

Temporary Coding

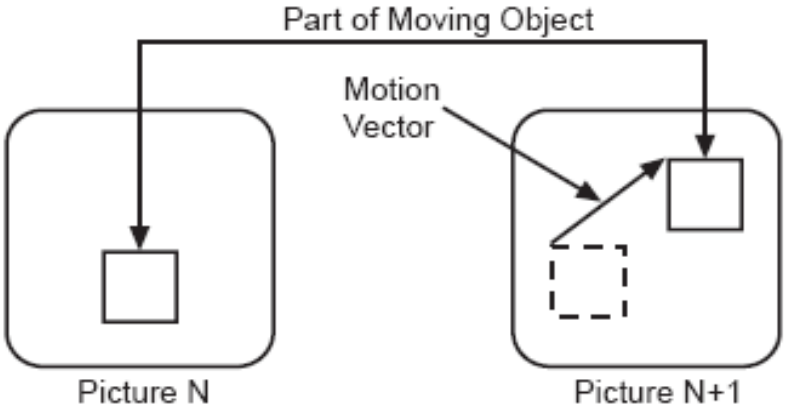
- **Full Search**
  - **31x31= 961 block matching per MB** (search window -15 ~ +15).

- **Three Step Search**
  - Step 1: An initial step size is picked. Eight blocks at a distance of step size from the center are picked for comparison.
  - Step 2: The step size is halved. The center is moved to the point with the minimum SAD (or MSE).
  - Steps 1 and 2 are repeated till the step size becomes 1.
  - **9+8+8=25 block matching per MB**



3-Step Search

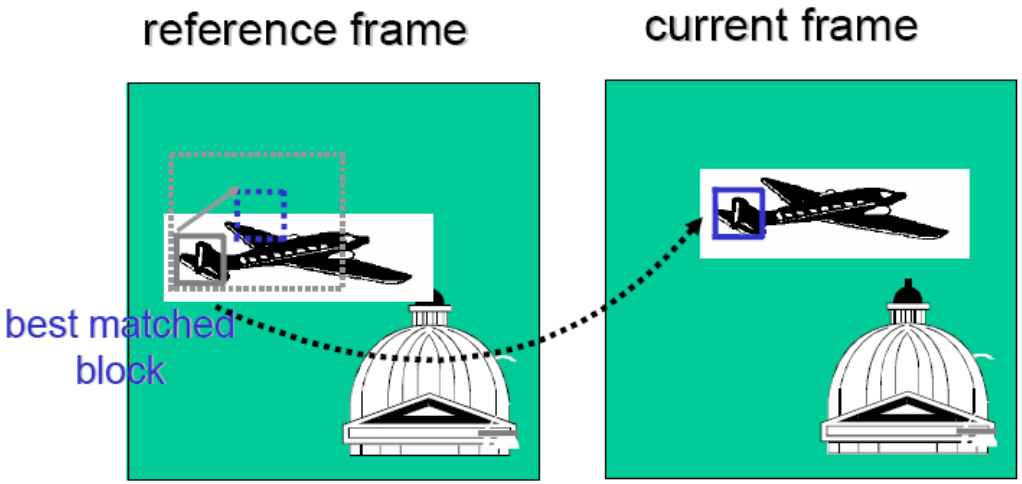
Temporary Coding



Actions

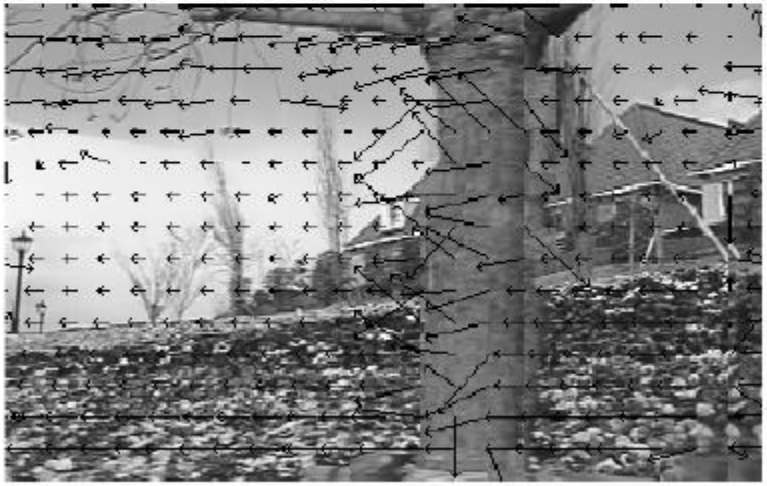
1. Compute Motion Vector
2. Shift data from picture N using vector to make predicted picture N+1
3. Compare actual picture with predicted picture
4. Send vector and prediction error

using spatial coding

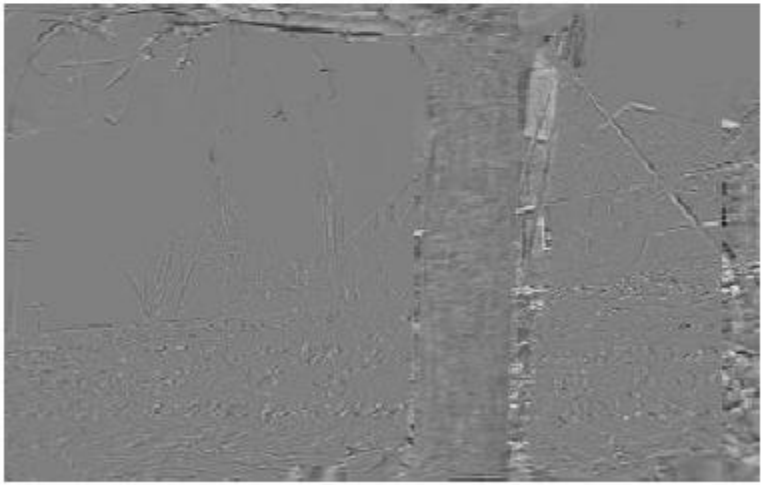


Frame 1

Frame 2



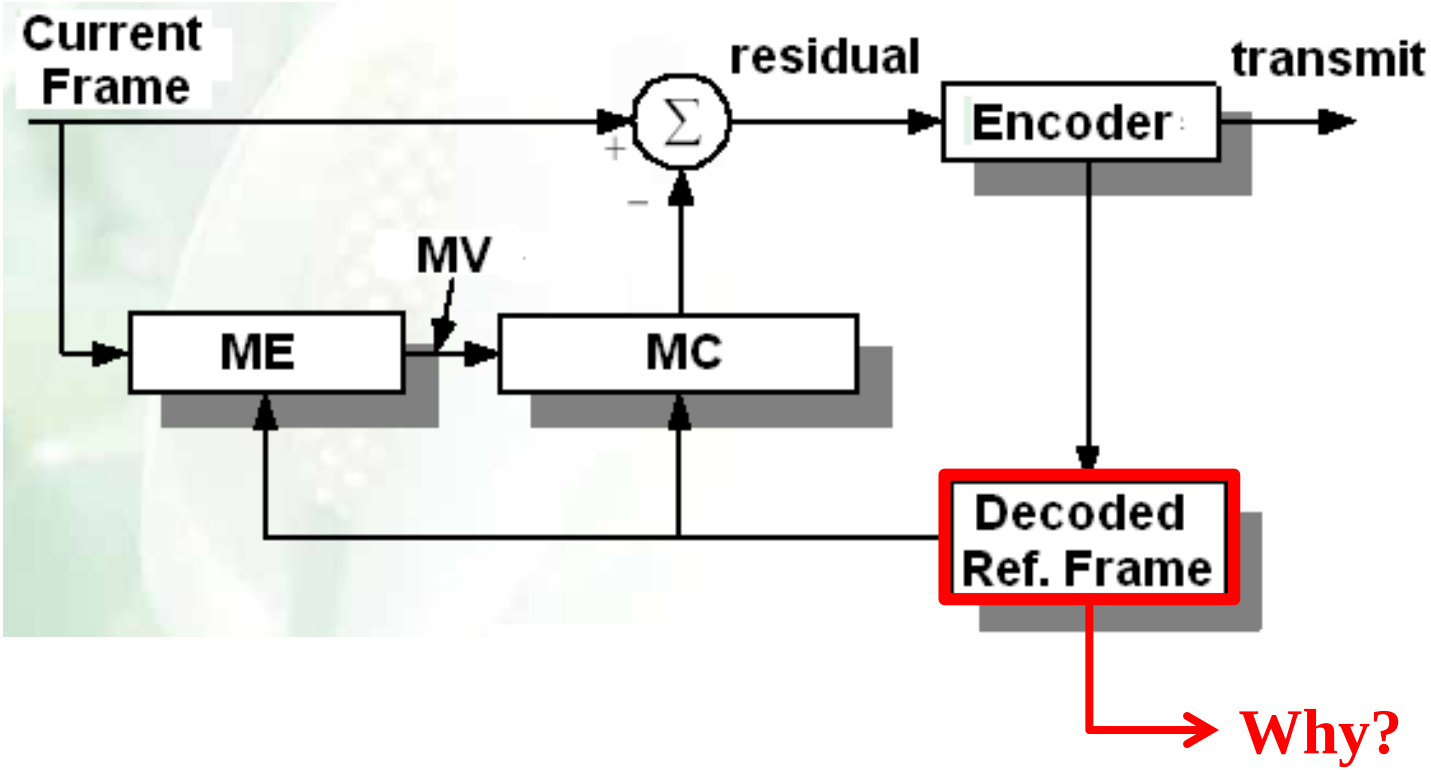
Frame2 with MV



MC prediction error frame

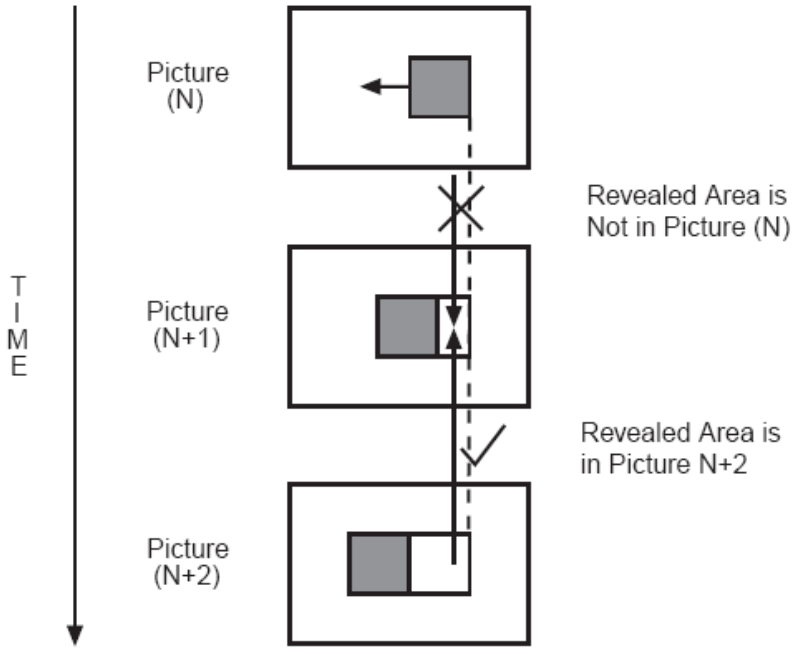
Temporary Coding

- **Encoder with Motion Estimation**



Temporary Coding

- **Bi-directional coding**
  - Consider when an object moves, it conceals the background at its leading edge and reveals the background at its trailing edge. The revealed background requires new data to be transmitted because no info. can be obtained from a previous picture.
  - Bi-directional coding allows motion-compensated data from an **earlier** or **later** picture, or even use an average of earlier and later data.



Temporary Coding

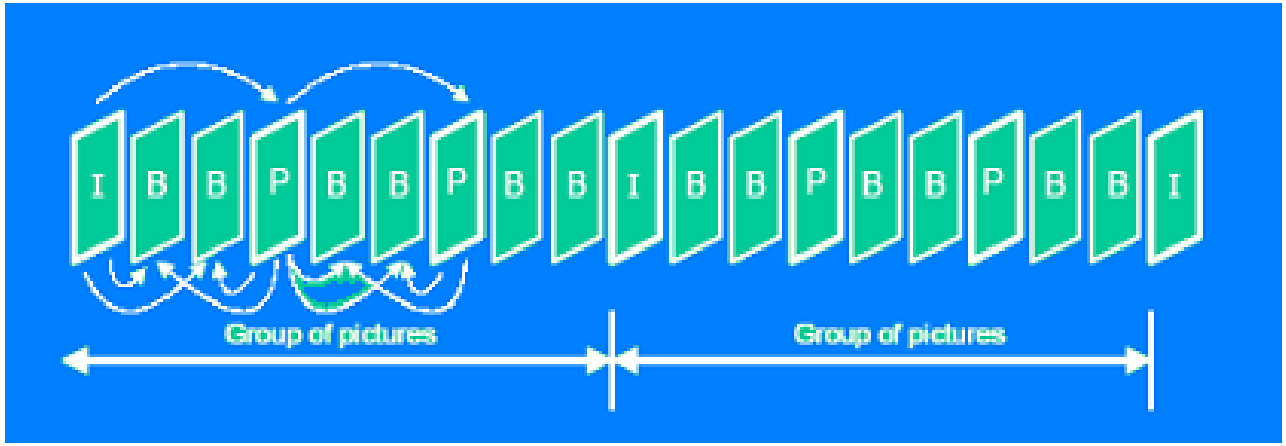
- **I, P, and B pictures**
  - **<u>I-Pictures</u>**
    - Intra-frame coded with 8x8 DCT
    - Arbitrary weighting matrix for coefficient quantization
    - Differential coding of DC-coefficients
    - Zig-zag-scan, run-length-coding, Entropy coding
    - ***Require more data, and thus transmitted less frequently.***
    - ***Allow users to switch channels, and arrest error propagation*.**
  - **<u>P-Pictures</u>**
    - Motion-compensated prediction from an encoded I- or P- picture
    - Half-pel accuracy of motion compensation, bilinear interpolation
    - One motion vector per macroblock
    - Differential coding of motion vectors
    - Coding of prediction error with 8x8-DCT, quantization, zig-zag-scan as in I-pictures
    - ***P pictures require roughly half the data of an I picture***

Temporary Coding

- **I, P, and B pictures**
  - **<u>B-Pictures</u>**
    - Motion-compensated prediction from two consecutive P- or I-pictures, **either**
      - *only forward prediction* (1 vector/macroblock), **or**
      - *only backward prediction* (1 vector/macroblock), **or**
      - *average of forward and backward prediction*
        = interpolation (2 vectors/macroblock)
    - Half-pel accuracy of motion compensation, bilinear interpolation
    - Coding of prediction error with 8x8-DCT
    - quantization, zig-zag-scan as in I-pictures
    - ***B picture typically require one quarter the data of an I picture***

Temporary Coding

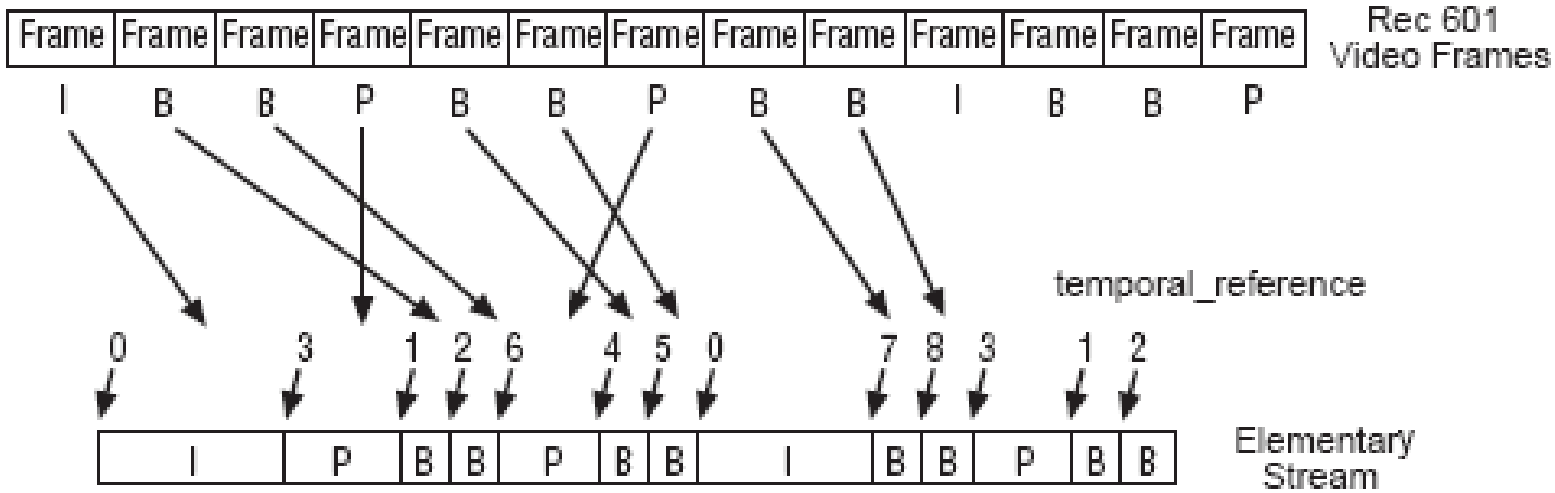
- **GOP (Group Of Picture)**
  - GOP begins with an I picture and ends before next I picture.
  - P pictures are spaced throughout, and the remaining are B pictures.
  - The GOP length is flexible. 12 or 15 pictures is common.



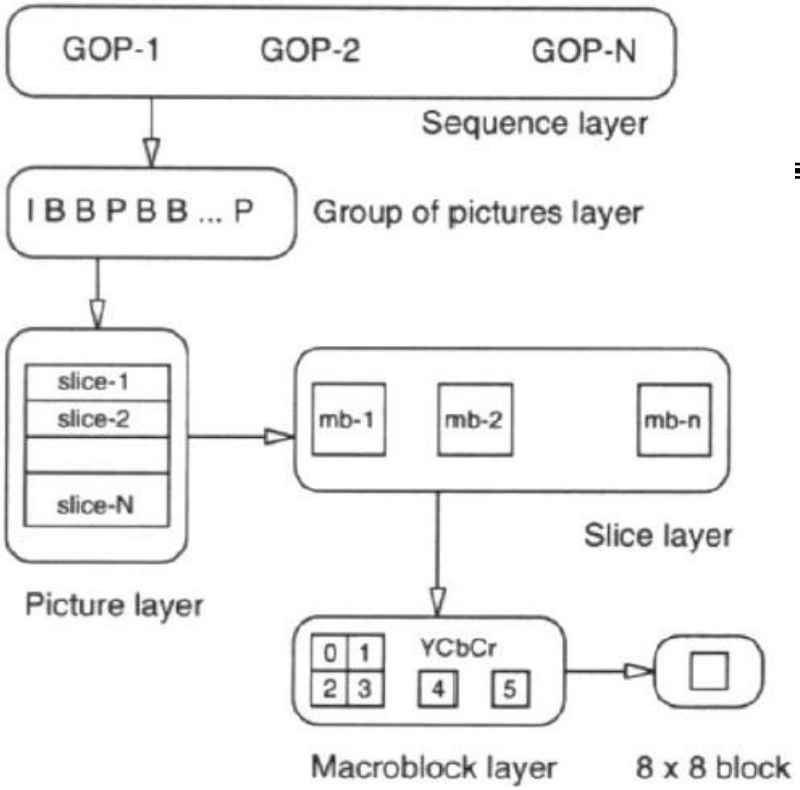
I = **Intra-Picture** Coding, allow random access, act as reference.
P = **Predictive** coding, can be referenced
B = **Bi-directional** coding, never be referenced

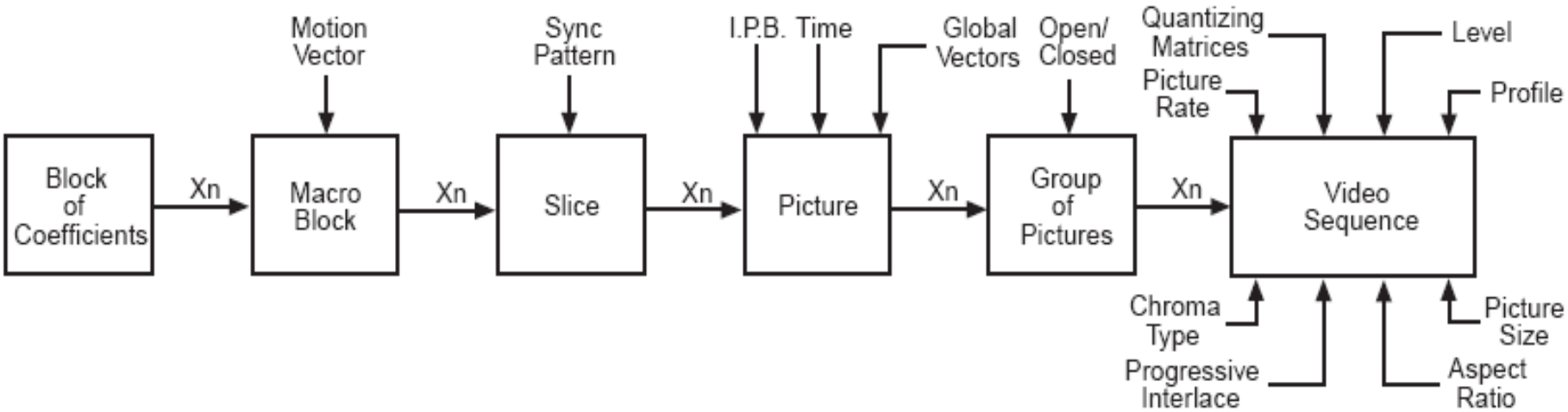
Temporary Coding

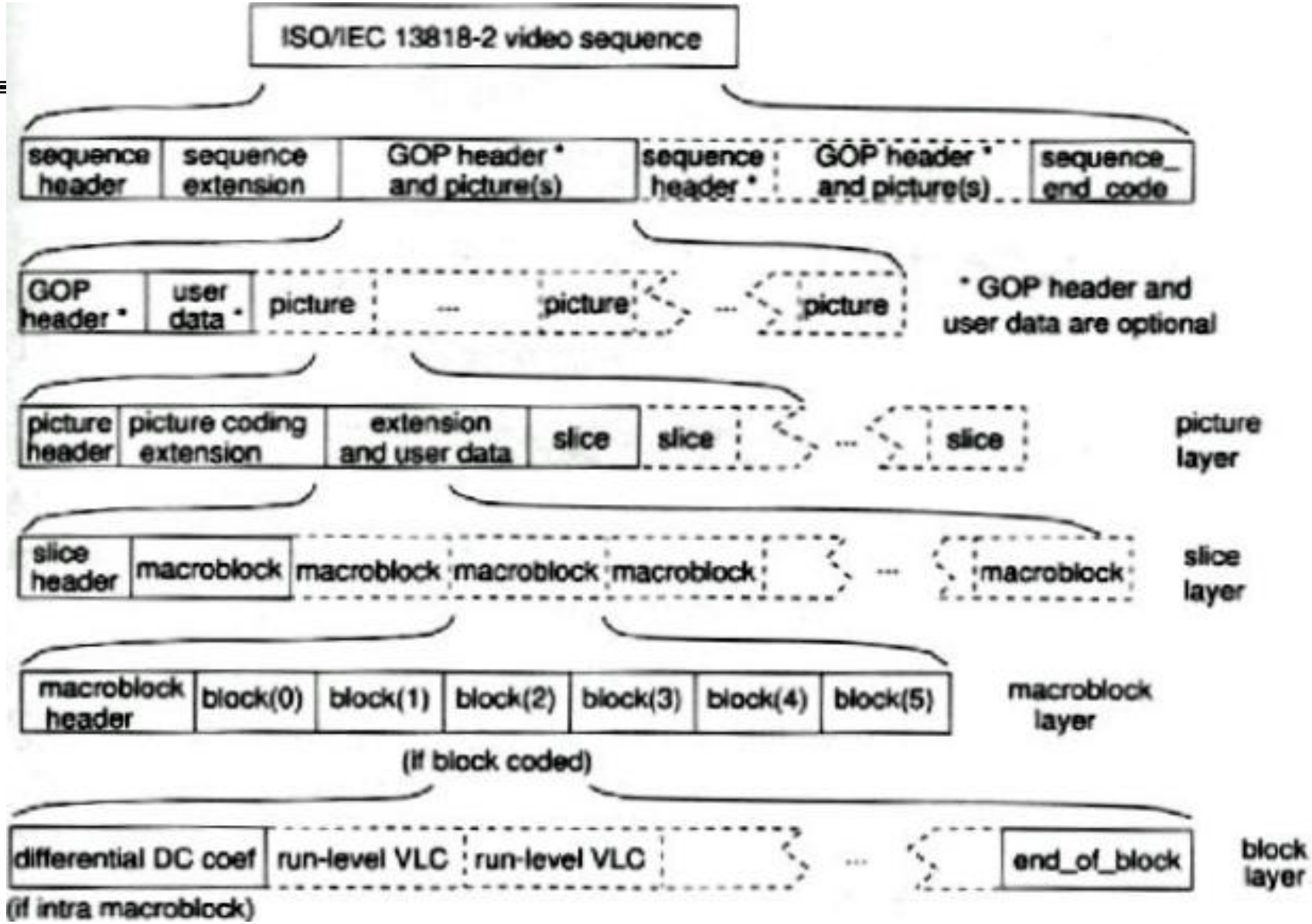
- **GOP (Group of Pictures)**
  - **Encoding/Decoding sequence v.s. Display sequence**
    - Since data for B pictures are from a future picture, that data must be available earlier at the decoder. This requires pictures to be sent out of sequence and temporarily stored.
    - 3 frame buffers are required (2ref + 1decoding frames)
  - **Open GOP vs. Closed GOP**



Video Elementary Stream






ISO/IEC 13818-2 video sequence
sequence header
sequence extension
GOP header * and picture(s)
sequence header *
GOP header * and picture(s)
sequence_ end_code
GOP header *
user data *
picture
picture
picture
* GOP header and user data are optional
picture header
picture coding extension
extension and user data
slice
slice
slice
picture layer
slice header
macroblock
macroblock
macroblock
macroblock
macroblock
slice layer
macroblock header
block(0)
block(1)
block(2)
block(3)
block(4)
block(5)
macroblock layer
(if block coded)
differential DC coef
run-level VLC
run-level VLC
end_of_block
block layer
(if intra macroblock)

Profile and Level

- **Profile and Level**
  - MPEG is applicable to a wide range of applications requiring different performance and complexity.
  - **Profile**: a subset of the entire coding repertoire
    - **Main profile:** I, P, B pictures, and 4:2:0 format
    - **Simple profile:** main profile without B pictures
    - **High profile:** 4:2:2 format for professional equipments.
    - **SNR and Spatial profiles:** scalability support for streaming app.
  - **Level**: a parameter like picture size or bit rate used with that profile.
    - **Low level:** 352x240, < 4 mbps, VCD
    - **Main level:** 720x480, < 15 mbps, DVD, SDTV
    - **High 1440 level:** double the definition compared to main level.
    - **High level:** 1920x1080 (16:9), < 80mbps, HDTV

Chapter 1: Source Coding- Video

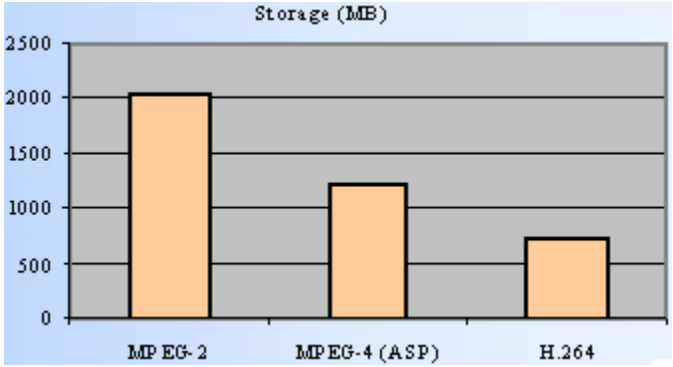
- **Video Coding – MPEG2**
- **Advanced Video Coding (H.264/AVC)**
- **High Efficient Video Coding (HEVC)**

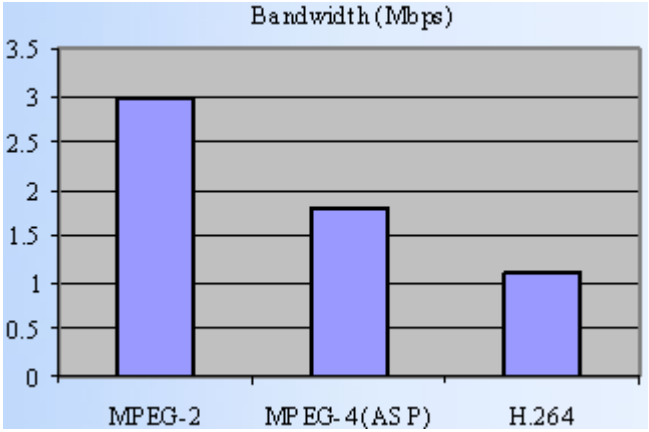
H.264 Introduction

- H.264 Performance
  - Complexity v.s. Efficiency

MPEG-4 AVC / H.264 Profiles	Target applications	complexity increase over MPEG-2	efficiency improvements over MPEG-2
Baseline Profile	low delay applications, video phone, mobile ...	2.5 X more	1.5 x better
Extended Profile	mobile, streaming, ...	3.5 X more	1.75 x better
Main Profile	interlaced video applications, broadcast, packaged media, ...	4.0 X more	2.0 x better

  - Comparison between mpeg2, mpeg4 ASP and H.264 for DVD quality video (90 min).



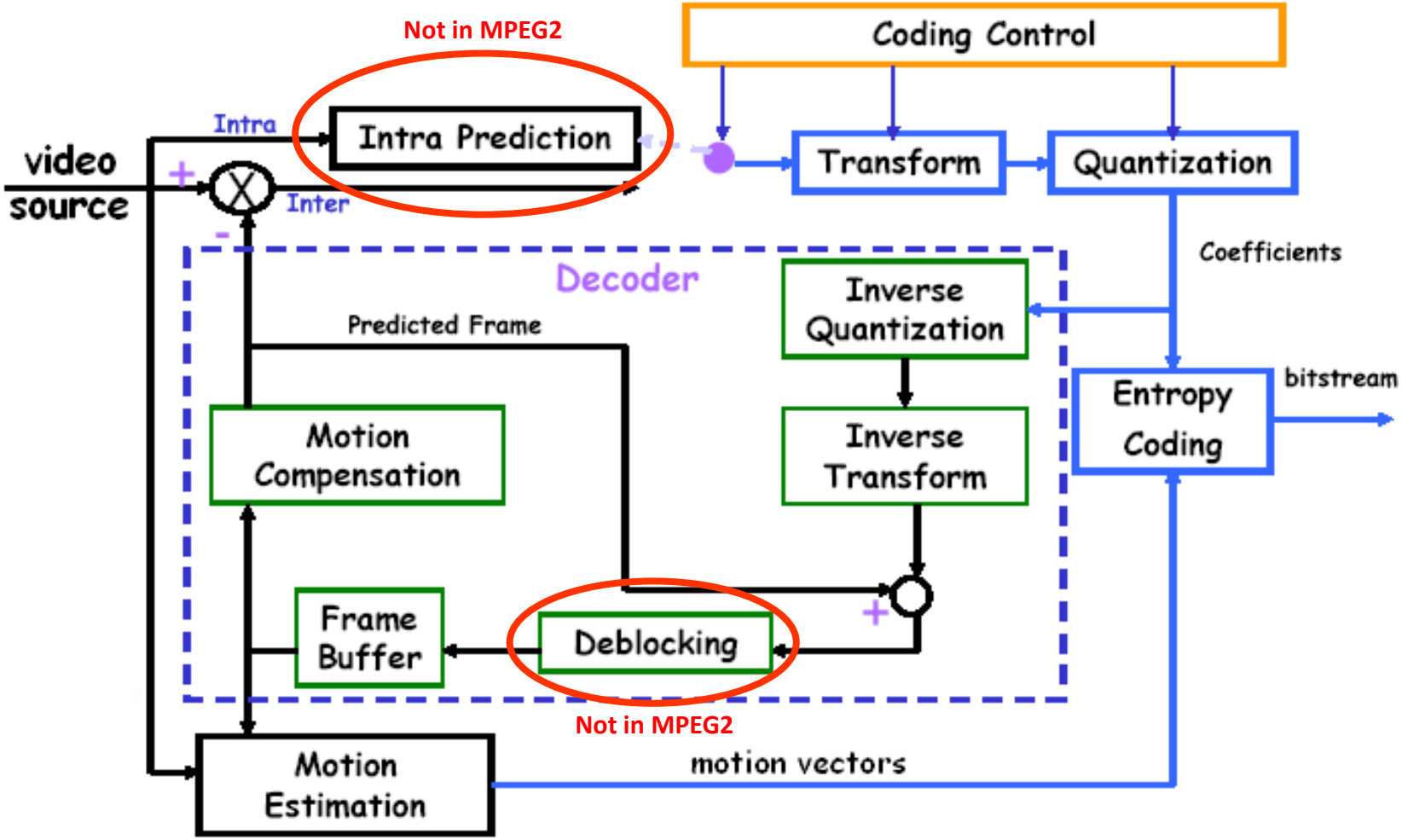


Overview of H.264

- Video coding is similar in spirit to other standards but with some important differences
- Key new features:
  - Intra-coding Prediction
  - Enhanced motion estimation
  - Integer block transform
  - Improved de-blocking filter
  - Enhanced entropy coding
- Average bit rate reduction of 50% compared to other standards

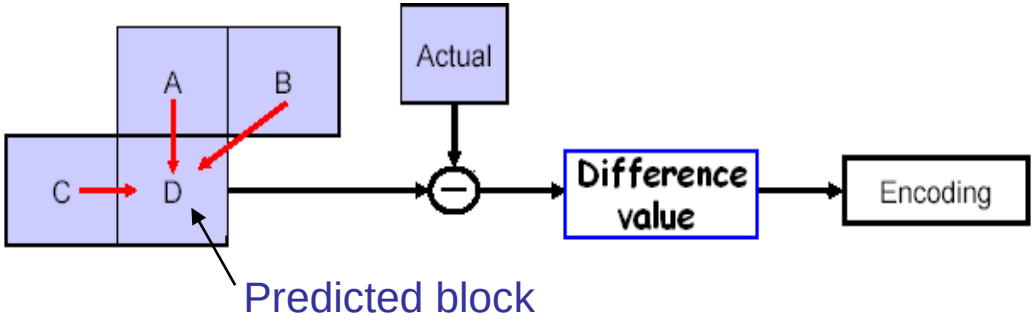
Video Coding System

H.264 Encoder



Intra-Prediction

- Spatial Prediction (Intra-prediction)



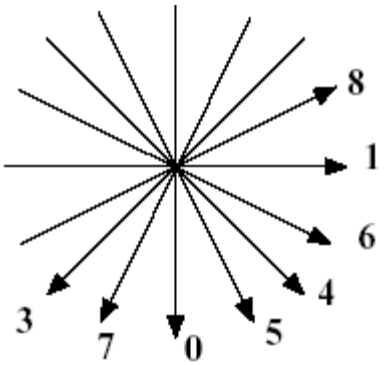
  - Luma prediction
    - 4×4 mode: 9 prediction modes
    - 16×16 mode: 4 prediction modes (for smooth image areas).
  - Chroma prediction
    - 8x8 mode: 4 prediction modes.
      - Cb and Cr use the same mode.

M	A	B	C	D	E	F	G	H
I	a	b	c	d				
J	e	f	g	h				
K	i	j	k	l				
L	m	n	o	p				

  - a – p is predicted from A – M
    - across slice boundaries is not allowed

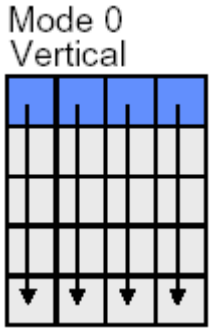
Intra-Prediction

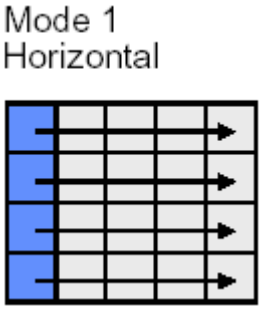
Luma Prediction (4x4)

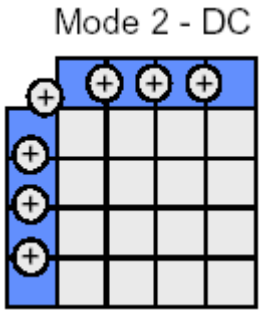


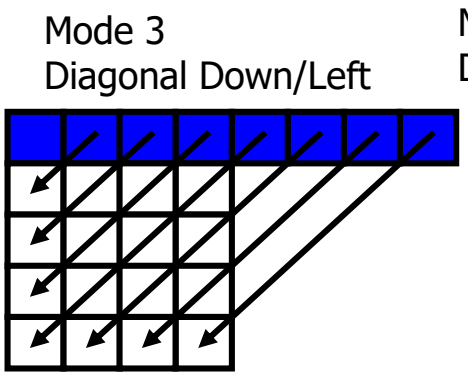
0	Vertical
1	Horizontal
2	DC
3	Diagonal Down Left
4	Diagonal Down Right
5	Vertical Right
6	Horizontal Down
7	Vertical Left
8	Horizontal Up

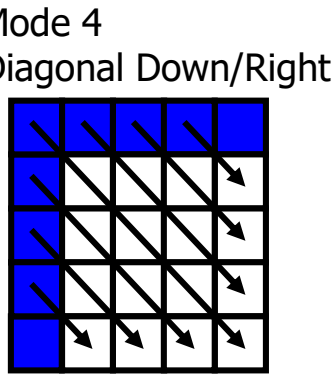
4×4 prediction modes

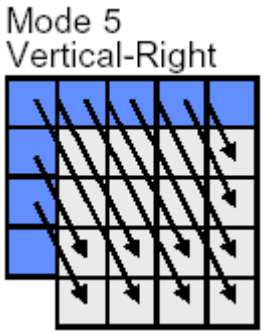


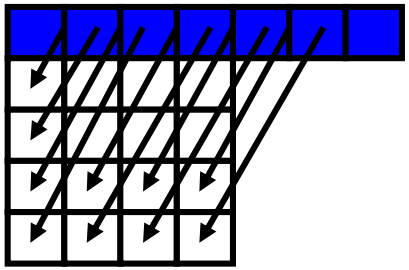


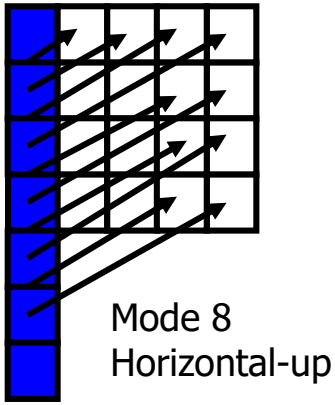












Intra-Prediction

Luma Prediction (4x4)

M	A	B	C	D	E	F	G	H
I	a	b	c	d				
J	e	f	g	h				
K	i	j	k	l				
L	m	n	o	p				

Mode 0: Vertical prediction mode

$$\begin{array}{ccccccccc} a & = & e & = & i & = & m & = & A \\ b & = & f & = & j & = & n & = & B \\ c & = & g & = & k & = & o & = & C \\ d & = & h & = & l & = & p & = & D \end{array}$$

Mode 1: Horizontal prediction mode

$$\begin{array}{ccccccccc} a & = & b & = & c & = & d & = & I \\ e & = & f & = & g & = & h & = & J \\ i & = & j & = & k & = & l & = & K \\ m & = & n & = & o & = & p & = & L \end{array}$$

Mode 2: DC prediction

If A, B, C, D, I, J, K and L are available, then

$$P = \frac{A+B+C+D+I+J+K+L+4}{8}$$

If only A, B, C and D exist, then

$$P = \frac{A+B+C+D+2}{4}$$ (ex: the top row)

if only I, J, K, and L exist, then

$$P = \frac{I+J+K+L+2}{4}$$ (ex:leftmost col)

If all the samples (A~D, I~L) are not available, then

P = (1 << ($BitDepth_Y$-1))

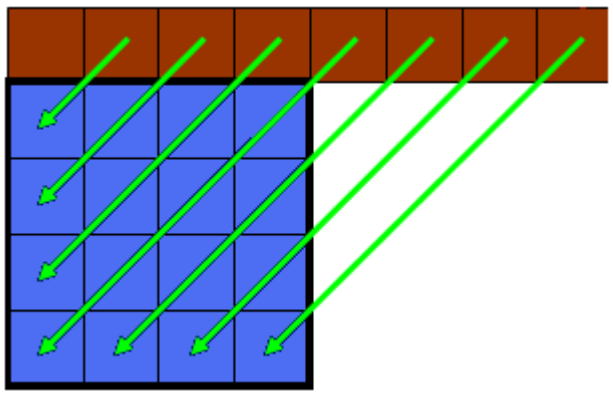
$BitDepth_Y$ is bit length of Luma Samples。

(ex:left-top corner)

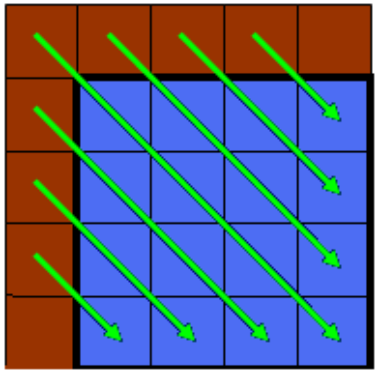
Intra-Prediction

Luma Prediction (4x4)

Mode 3: Diagonal Down Left



Mode 4: Diagonal Down Right



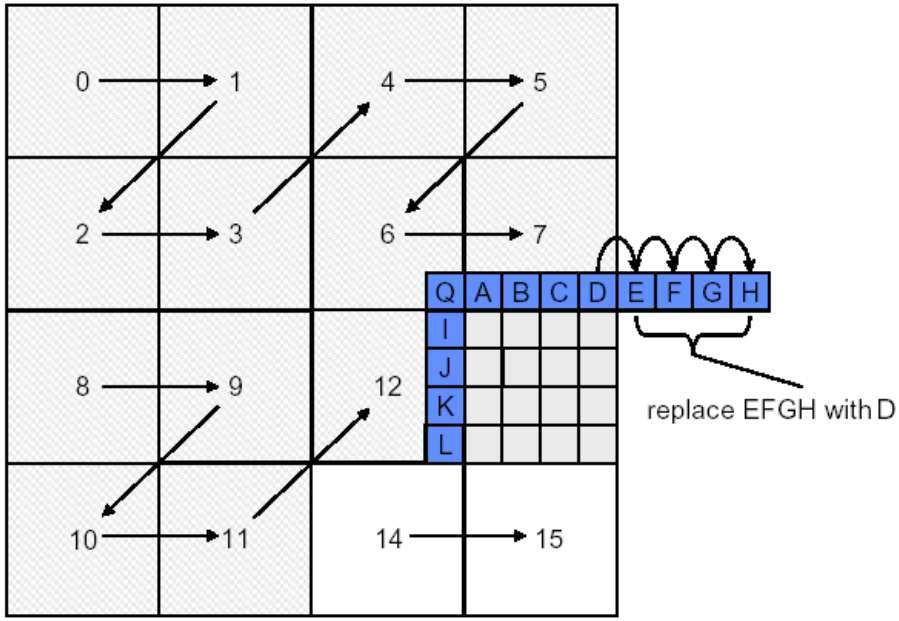
$$
\begin{aligned}
a &= \frac{A+2B+C+2}{4} \\
b=e &= \frac{B+2C+D+2}{4} \\
c=f=i &= \frac{C+2D+E+2}{4} \\
d=g=j=m &= \frac{D+2E+F+2}{4} \\
h=k=n &= \frac{E+2F+G+2}{4} \\
l=o &= \frac{F+2G+H+2}{4} \\
p &= \frac{G+3H+2}{4}
\end{aligned}
$$

M	A	B	C	D	E	F	G	H
I	a	b	c	d				
J	e	f	g	h				
K	i	j	k	l				
L	m	n	o	p				

Intra-Prediction

Luma Prediction (4x4)

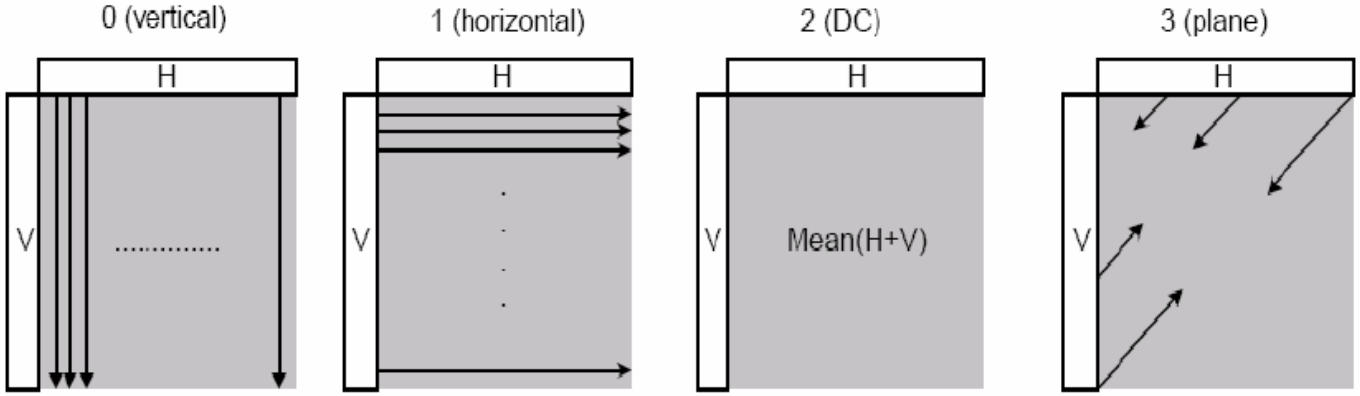
- Boundary condition



Luma Prediction (16x16)

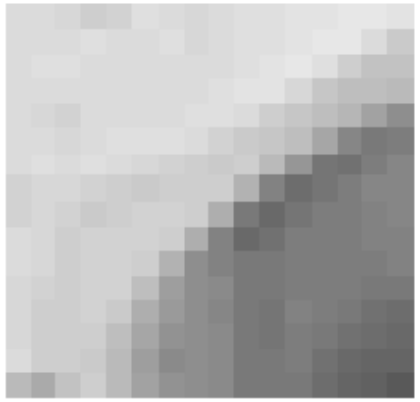
0	Vertical
1	Horizontal
2	DC
3	Plane

16 × 16 prediction modes

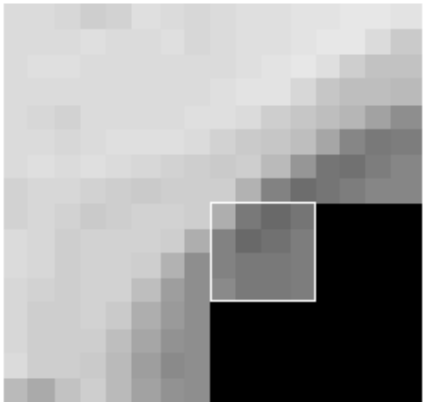


Intra-Prediction

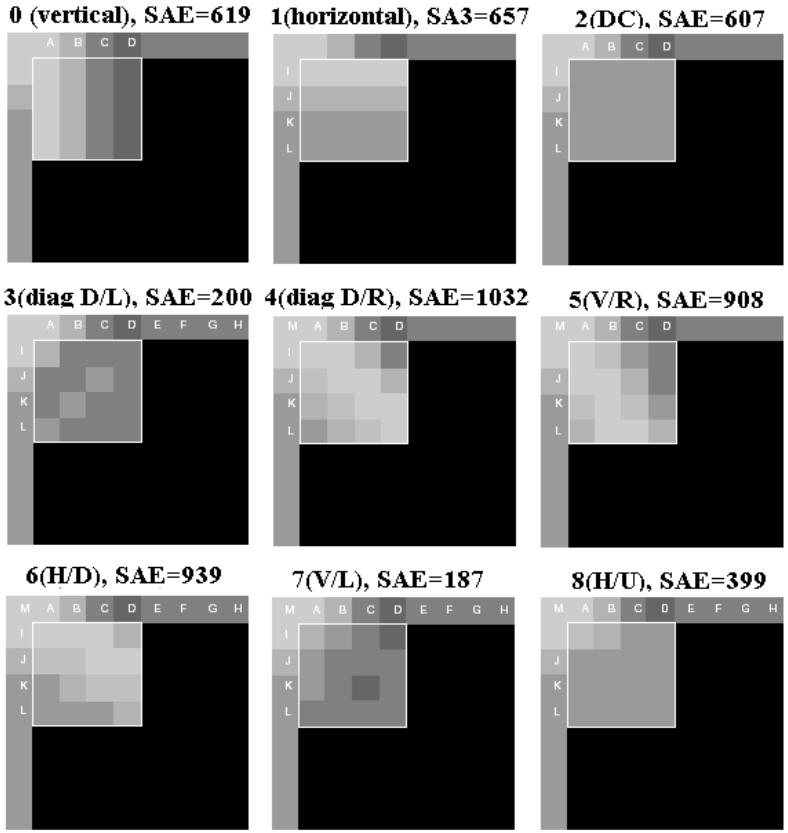
Original macroblock



4x4 luma block to be predicted



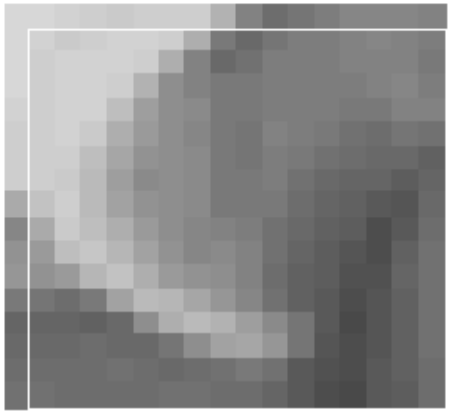
Luma Prediction (Intra-4x4)

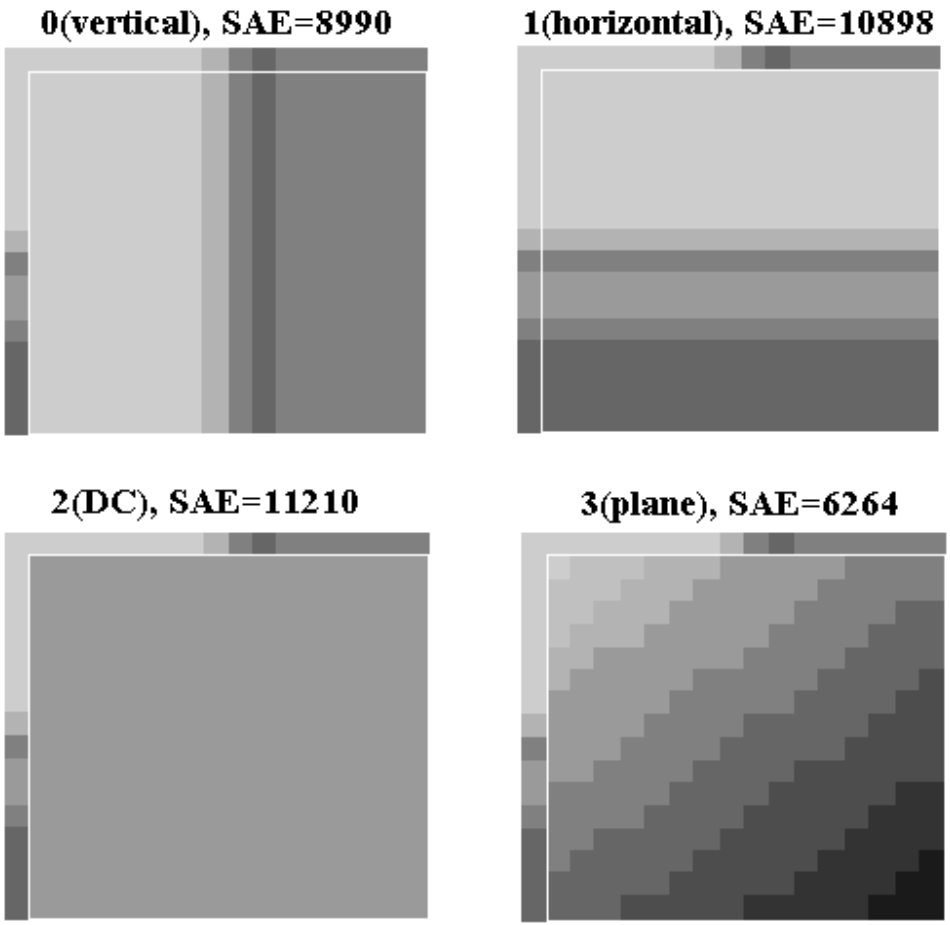


SAE : Sum of Absolute Errors

Intra-Prediction

Luma Prediction (Intra-16x16)

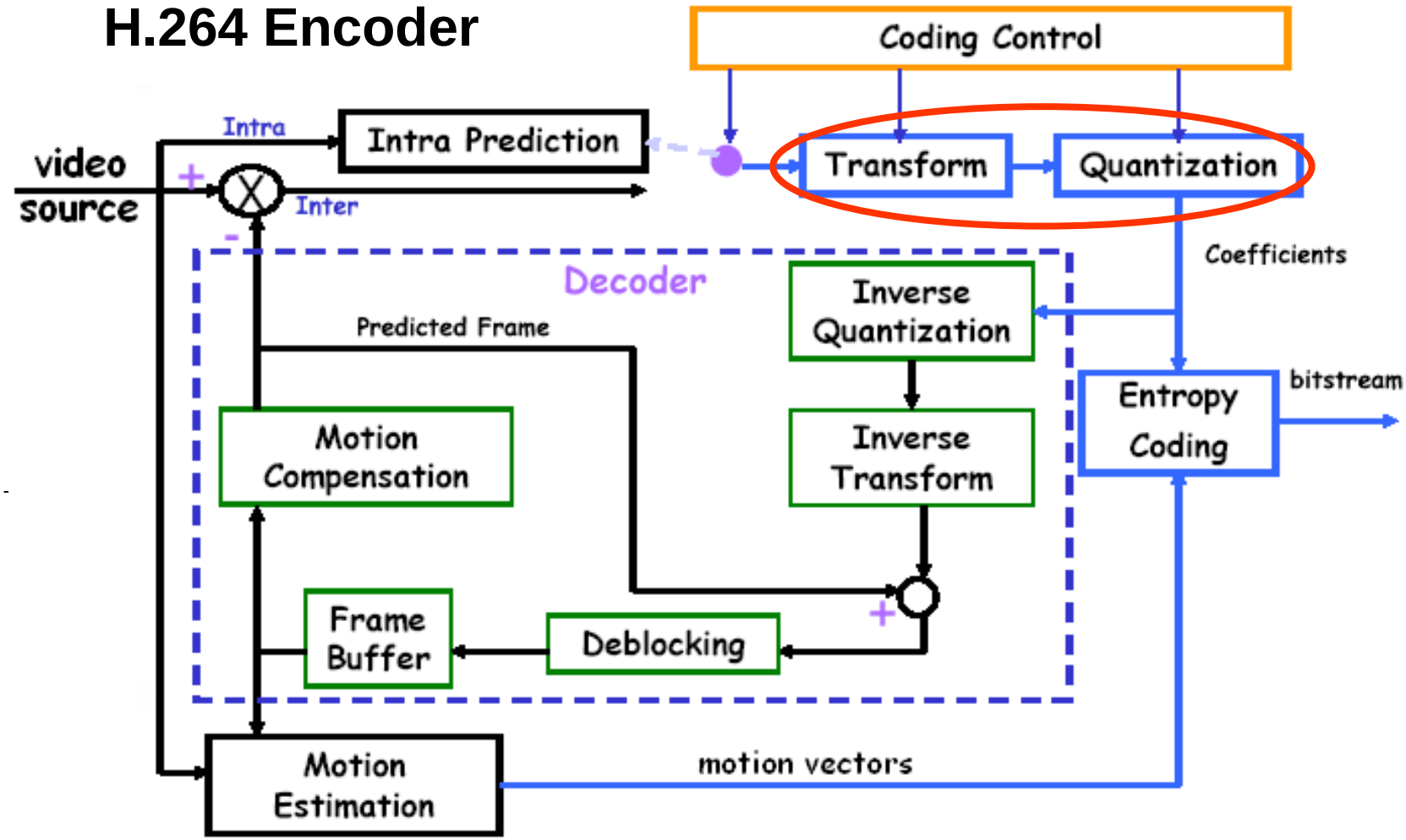




SAE : Sum of Absolute Errors

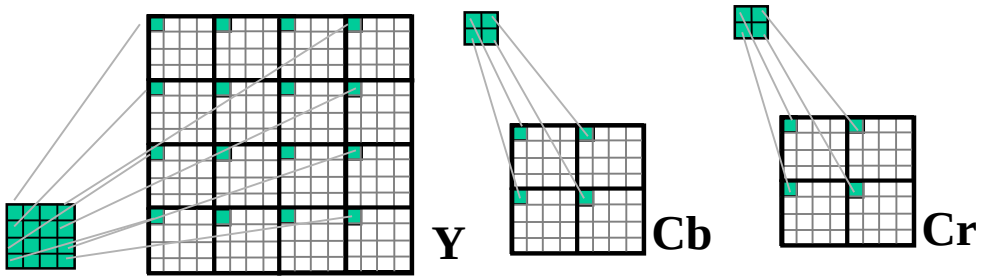
Video Coding System

H.264 Encoder



Transform

- Integer Transform
  - Recall: MPEG-1, MPEG-2, MPEG-4 and H.263
    - use DCT transform on 8x8 blocks
  - H.264 uses three integer transforms:
    - DCT-like integer transform for 4x4 block of residual data
    - Hadamard transform for 4x4 luma DC coeffs in intra-16x16 mode.
    - Hadamard transform for 2x2 chroma DC coeffs.
  - Advantages of Integer Transform
    - Reduce computation complexity (only adds and shift required).
    - All operations are integer arithmetic without loss of accuracy.
    - Mismatch between encoder and decoder will not occur.



Quantization

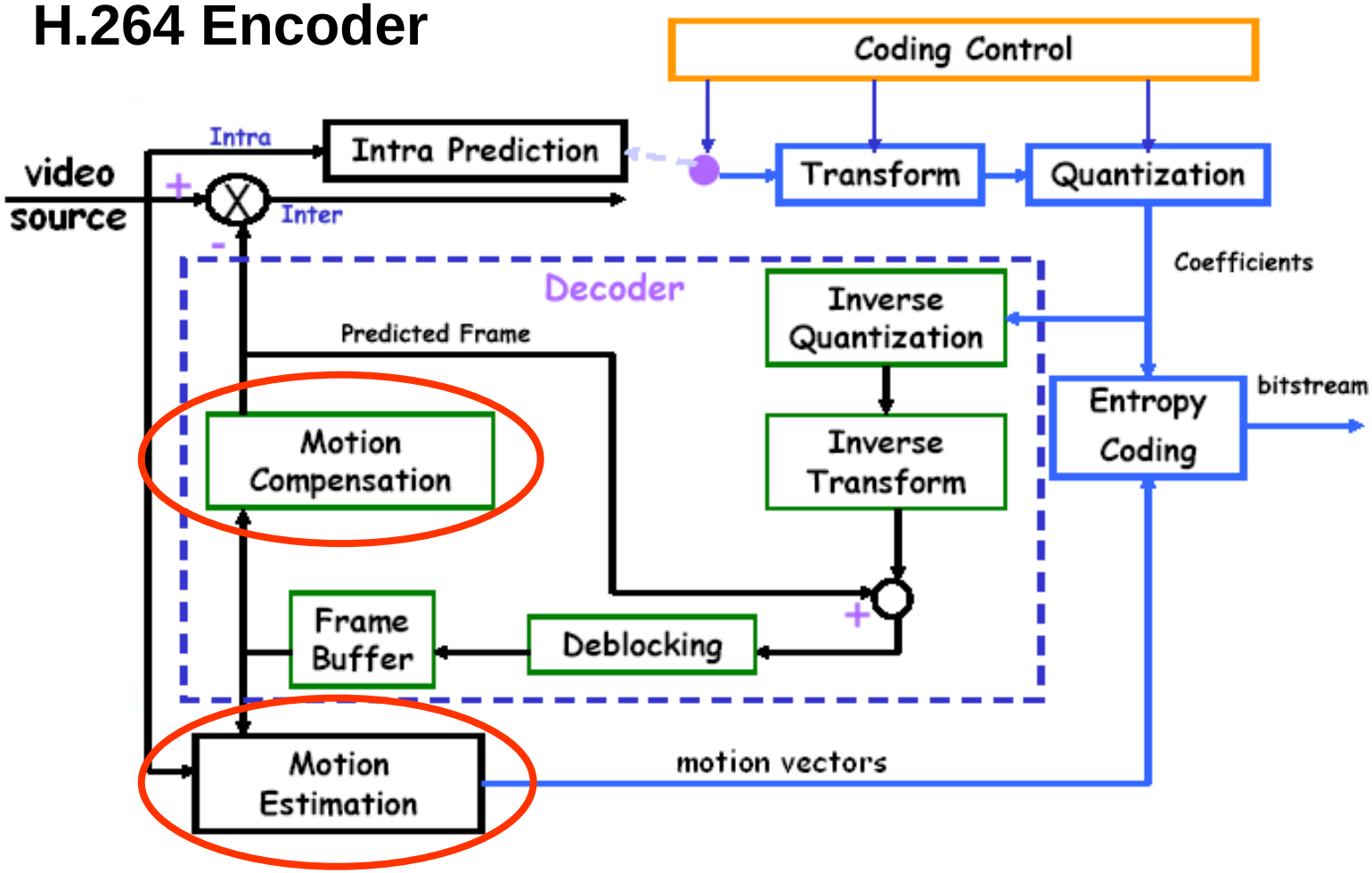
- **Quantization table**
  - Zij = round(Yij / Qstep)
    - where Yij is a coeffs after transform, Zij is a quantized coeff.
  - 52 Qsteps in total, indexed by QP (Quantization Parameter).
  - Qstep increases 12.5% for increment of 1 in QP.
  - Double in Qstep for every increment of 6 in QP.
  - The wide range of step sizes makes an encoder flexible to control the trade-off between bit-rate and quality.
  - One of 52 QP is selected for each macroblock..

QP	0	1	2	3	4	5	6	7	8	9	10	11	12	….
QStep	0.625	0.6875	0.8125	0.875	1	1.125	1.25	1.375	1.625	1.75	2	2.25	2.5	….
QP	…	18	…	24	…	30	…	36	…	42	…	48	…	51
QStep		5		10		20		40		80		160		224

Quantization step sizes in H.264

Video Coding System

H.264 Encoder

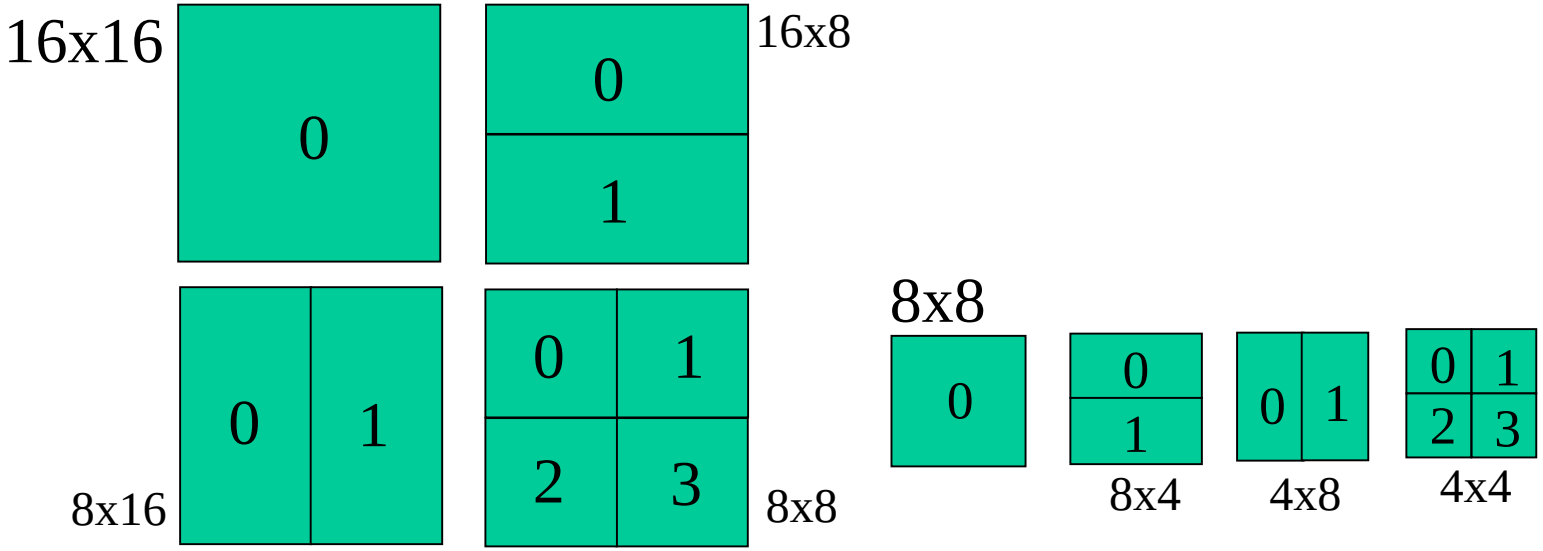


Inter-Coding

- Inter-Prediction
  - For H.264, the major differences from earlier standards include
    - support for a range of block sizes (down to 4x4)
    - support for sub-pixel motion vectors (1/4 pixel in luma component)
    - support for multi-picture motion-compensated prediction

Inter-Coding: Variable block size

- **Various Block Size**
  - A 16x16 luma macroblock can be subdivided
    - 16x16, 16x8, 8x16, 8x8 (known as macroblock partition)
    - If 8x8 mode is chosen, each of the four 8x8 MB partitions can be further split in 4 ways: 8x8, 8x4, 4x8, 4x4 (sub-partition).



Inter-Coding: Variable block size

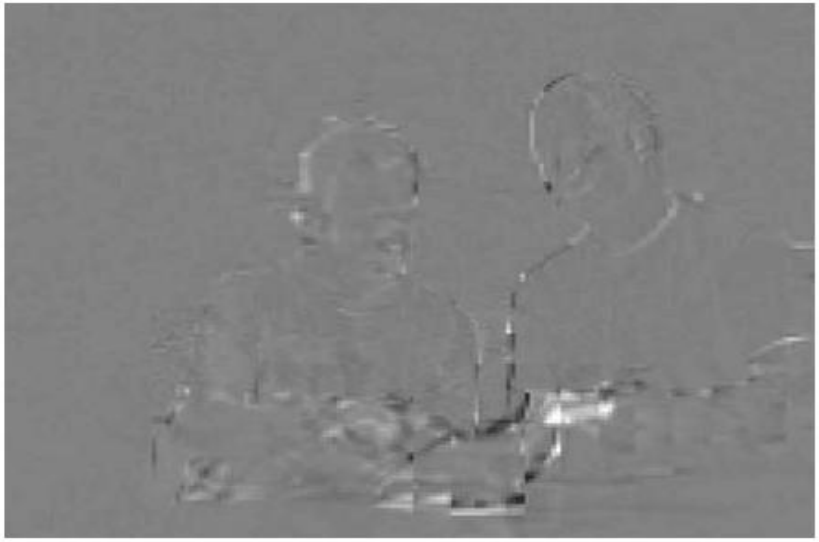
Various Block Size

- The choice of partition size
  - Large partition size (e.g. 16x16, 16x8, 8x16):
    - Few bits are required for motion vectors and type of partition.
    - More bits for motion compensated residual. It may contain a significant amount of energy in frame areas with high detail.
  - Small partition size (e.g. 8x4, 4x4, etc.):
    - Few bits for motion compensation residual.
    - More bits required for motion vectors and type of partition(s).
  - Chroma partition size
    - Chroma partition have half the horizontal and vertical resolution (e.g., an 8x16 partition in luma corresponds to a 4x8 in chroma)
    - The horizontal and vertical components of each motion vector (one per partition) are halved when applied to the chroma blocks.

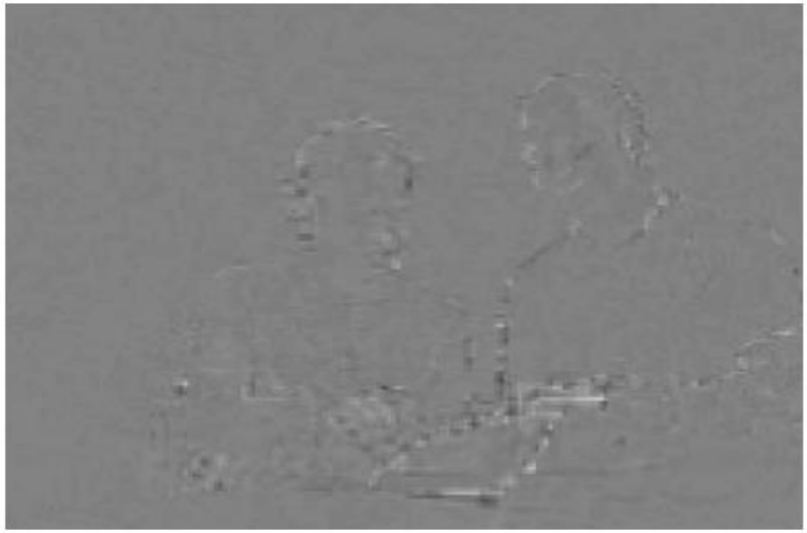
Inter-Coding: Variable block size

- **The Choice of Partition Size**

16 x 16 ➔



8 x 8

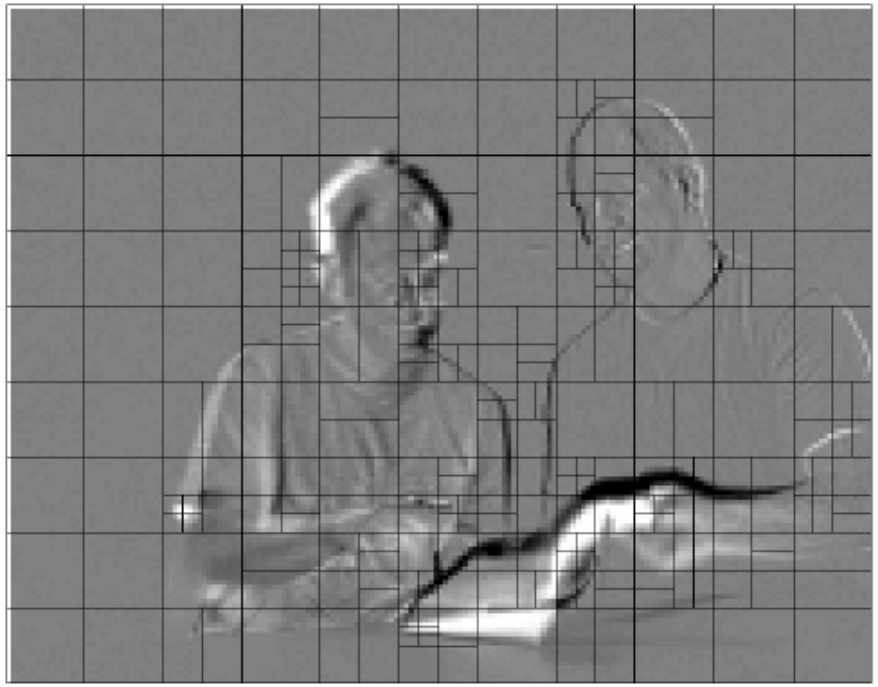


4 x 4

Inter-Coding: Variable block size

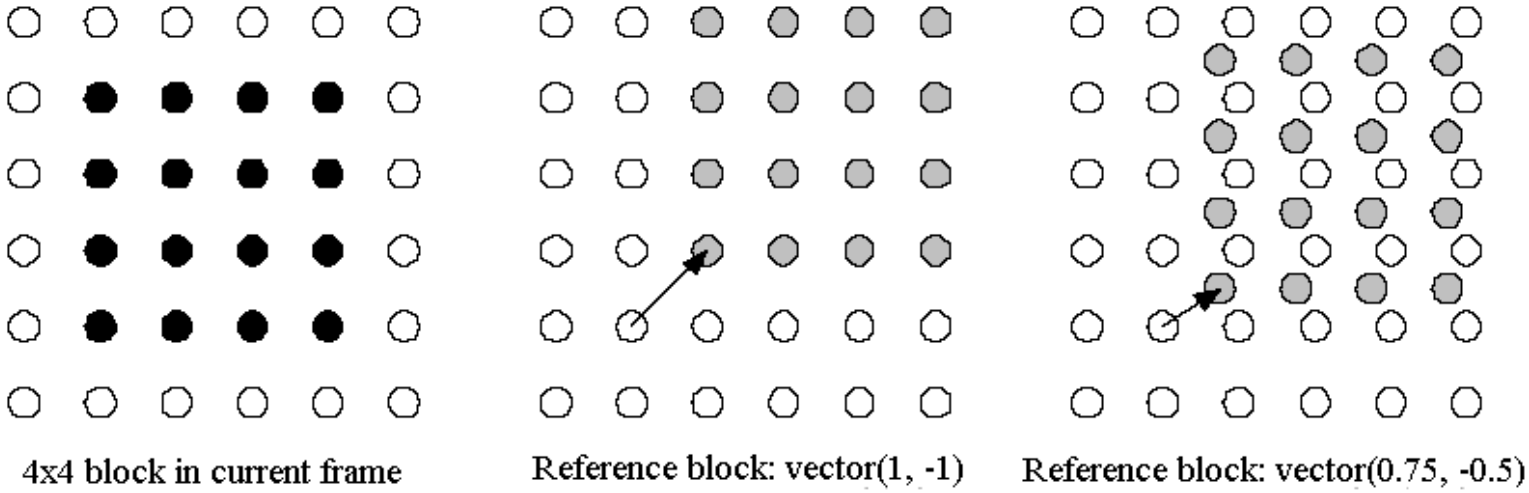
- Best partition size
  - The size that minimizes the coded residual and motion vectors.
  - A large partition size is appropriate for homogeneous areas of picture, and a small partition size is good for detailed areas.
  - H.264 encoder selects the “best” partition size for each part of the frame.

optimum choice of partitions
(without MC)



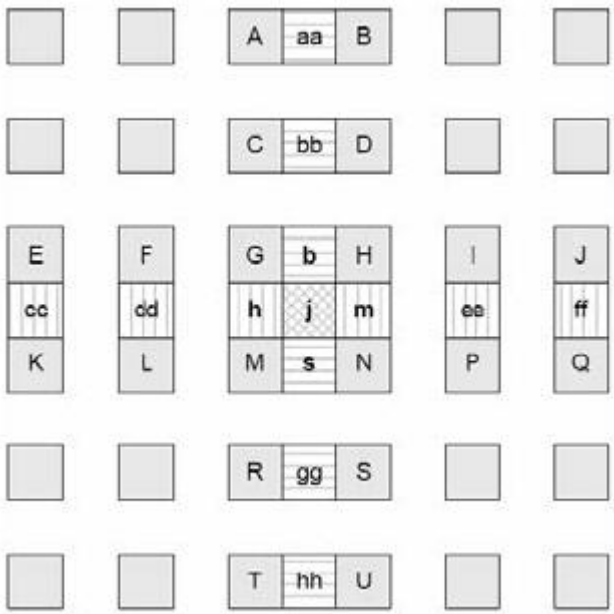
Inter-Coding: Sub-Pixel MV

- **Sub-Pixel Motion Vector**
  - In H.264, the offset between the two areas can have half- or quarter- pixel resolution.
  - Since the luma and chroma samples at sub-pixel positions do not exist in the reference picture, it is necessary *to create them using interpolation from nearby image samples*.
  - Sub-pixel motion compensation provides better compression than integer-pixel, at the cost of increased complexity.



Inter-Coding: Sub-Pixel MV

Half- Pixel Interpolation



b = round((E-5F+20G+20H-5I+J)/32)

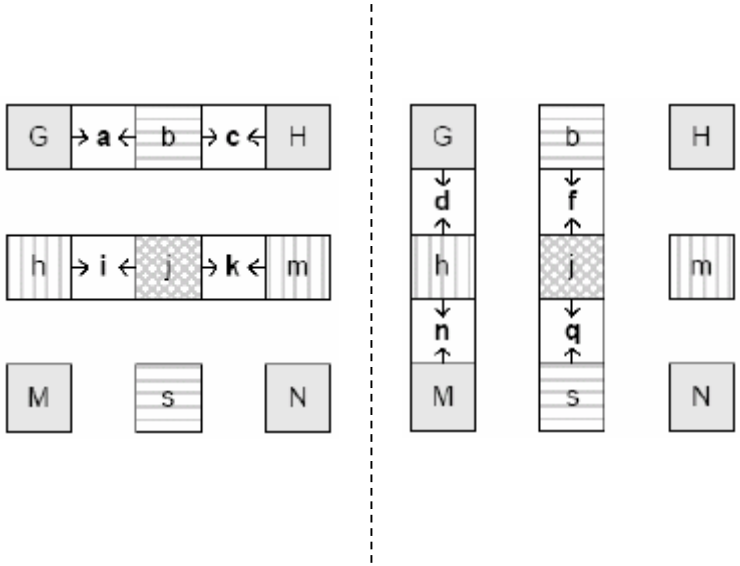
h = round((A-5C+20G+20M-5R+T)/32)

j = round((cc-5dd+20h+20m-5ee+ff)/32)

or round((aa-5bb+20b+20s-5gg+hh)/32)

6-tap FIR: (1, -5, 20, 20, -5, 1)/32

Quarter Pixel Interpolation



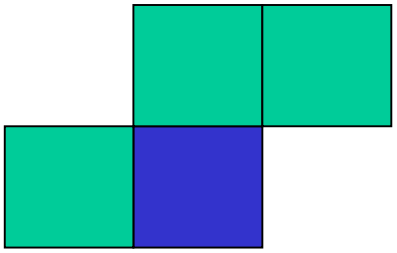
a = round((G+b)/2)

d = round((G+h)/2)

Bilinear interpolation

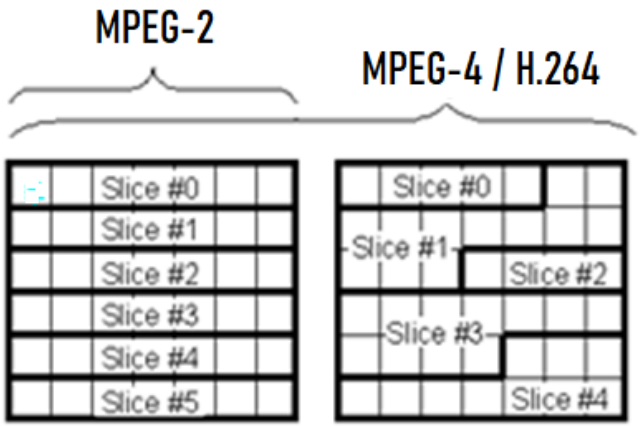
Inter-Coding: Sub-Pixel MV

- Motion Vector Prediction
  - Motion vectors for neighboring partitions are often highly correlated so each motion vector can be predicted from vectors of nearby partitions.
  - A predicted vector indicator, MVp, and the difference between the current vector and predicted vector, MVD, is encoded and transmitted.
  - The “basic” predictor is the median of the motion vectors of macroblock partitions or sub-partitions immediately above, diagonally above and to the right, and immediately left of the current partition (The predictor is modified if some of them are not available.)

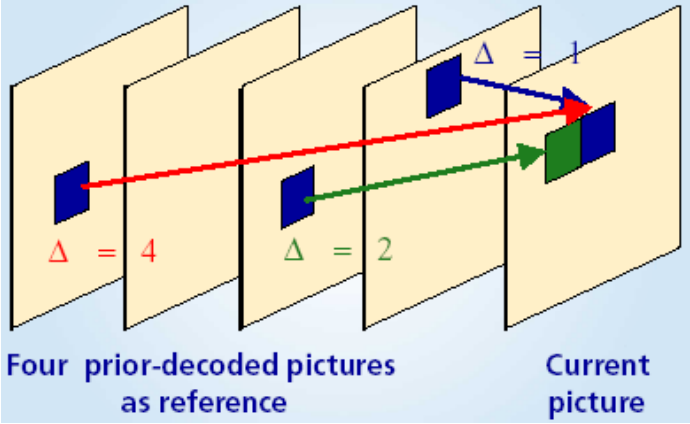


Inter-Coding: multi-frame prediction

- Slices: a sequence of MBs. Slices are self-contained. MBs within a slice depend on each other.
- Slice types in H.264
  - I-slice: intra-prediction
  - P-slice: intra- or inter- (one MV)
  - B-slice: intra- and inter- (two MVs)

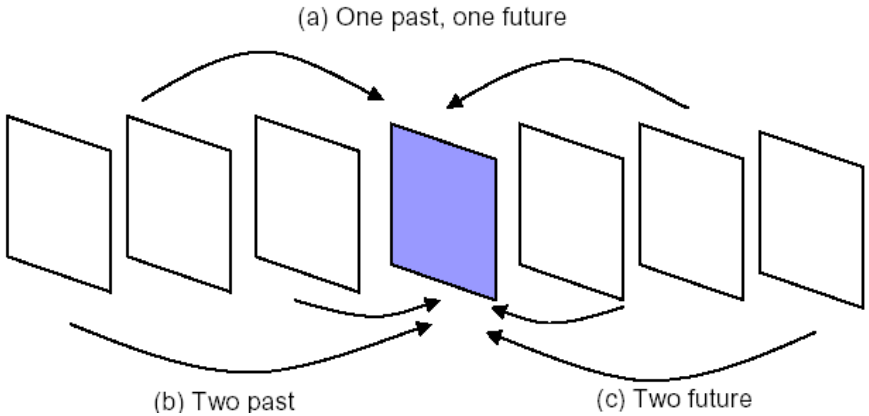


- Multi-picture motion-compensated prediction in P-slice
  - Improve the performance
  - Increase codec complexity
  - Increase size of frame buffer



Inter-Coding: multi-frame prediction

- Bi-Direction Prediction in B-slice
  - B-slices use two reference picture buffers. The one before current picture is called *list1*, and the other is called *list2* buffer.
    - **Forward prediction**: the prediction is from pictures in *list1* buffer.
    - **Backward prediction**: the prediction is from pictures in *list2* buffer.
    - **Bi-direction**: the prediction is from a weighted average of motion-compensation of *list 0* and *list 1*.
  - Each partition/sub-partition can use different prediction method separately.



Inter-Coding: multi-frame prediction

- How to select motion vector, reference frame, and block mode?
  - Recall: In MPEG2, MV selection is based on SAD or MAD.
  - In H.264, based on rate-distortion optimization (**RDO**).
- **RDO** in JM (the reference software of H.264)
  - Motion Estimation RDO (to determine the best motion vector)
    - **J = D + λR(mvd, ref)**

    Distortion: SAD between current MB and *predicted MB*.

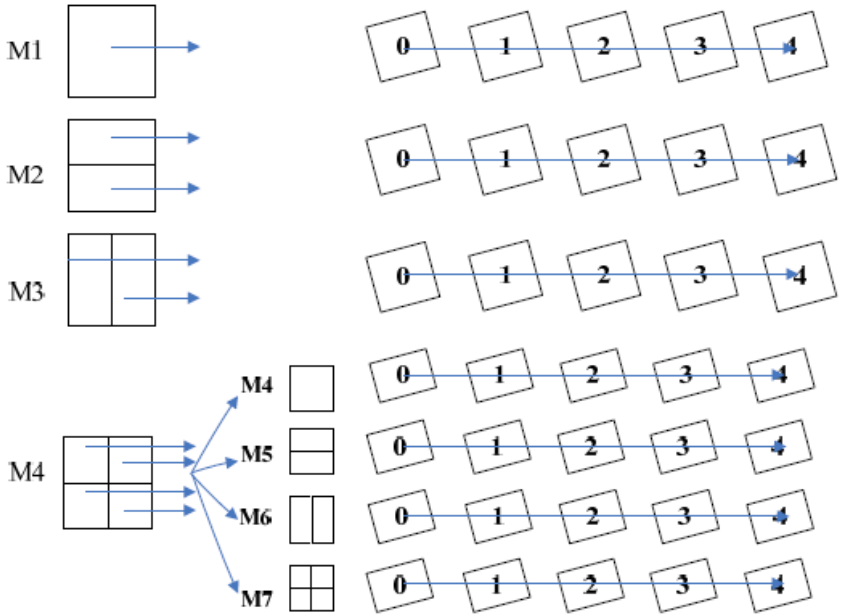
    Bit-Rate: Bit-rate by coding mvd and ref-frame index
  - Mode Decision RDO (to determine the best block size)
    - **J = D + λR(mvd, ref-index, mode, qDCTcoeff)**

    Distortion: SAD between current MB and *reconstructed MB*.

    Bit-Rate: Bit-rate by coding mvd, ref-frame index, block mode, qDCTcoeff.
    - Need to do Transform, Quantization, de-Q, inverse T

Inter-Coding: multi-frame prediction

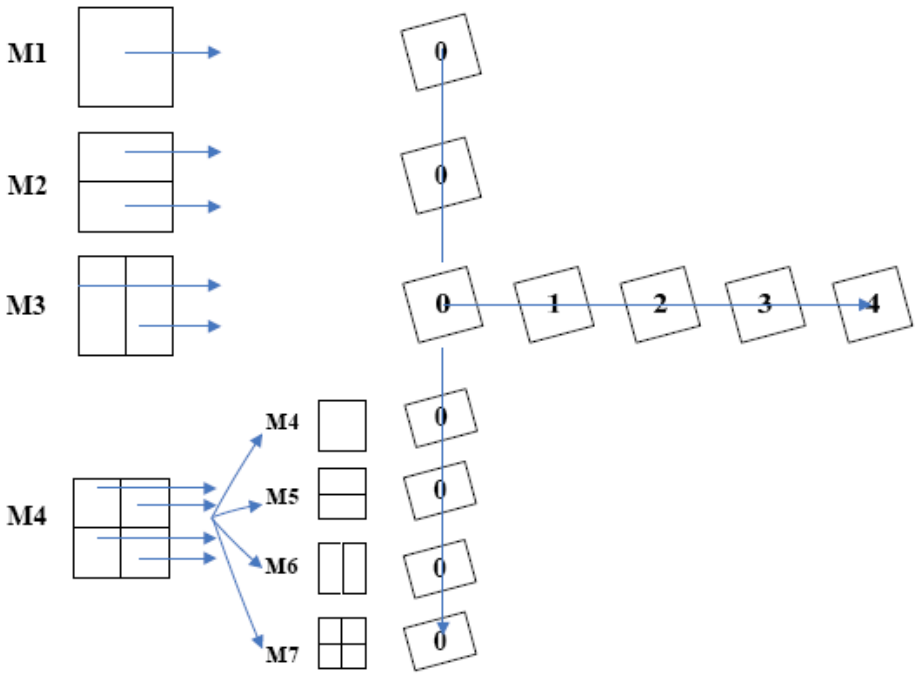
Motion Estimation in JM
(JM: H.264 reference software)



- For each mode, perform multi-reference-frame motion estimation
  - motion estimation RDO to choose best MV and ref. frame.
- Among the best MV and ref. frame of each mode
  - mode decision RDO determines the best mode (M1~M7)

Inter-Coding: multi-frame prediction

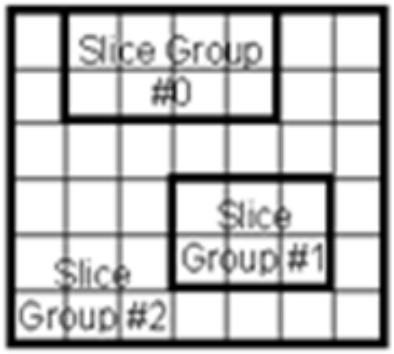
Fast Motion Estimation
(an example)



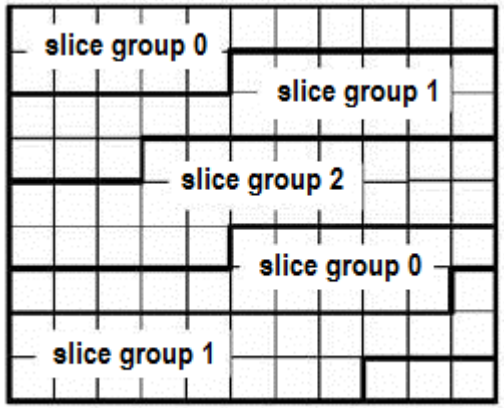
- For each block mode (M1~M7), do single reference-frame motion estimation
  - use motion estimation RDO to choose MV and ref. frame.
- Mode decision RDO determines the best block mode
- For the best block mode, do multiple reference-frame motion estimation
  - use motion estimation RDO to choose the best MV and ref. frame.

Inter-Coding: FMO

- **Slice group**
  - It contains 1 to several slices that are transmitted together.
  - Different slice groups are transmitted separately.
  - If a slice group is lost, other correctly-received slice groups can be used for error concealment.
- **FMO** (Flexible Macroblock Ordering)
  - It specifies macroblocks in a picture to one or several slice groups.
  - Seven FMO Types in H.264
    - Interleave
    - Disperse
    - Foreground with left-over
    - Box-out (counter box-out)
    - Raster scan (reverse)
    - Wipe right (left)
    - User defined

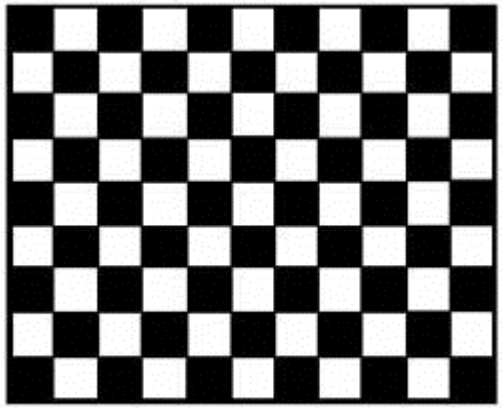


Inter-Coding: FMO



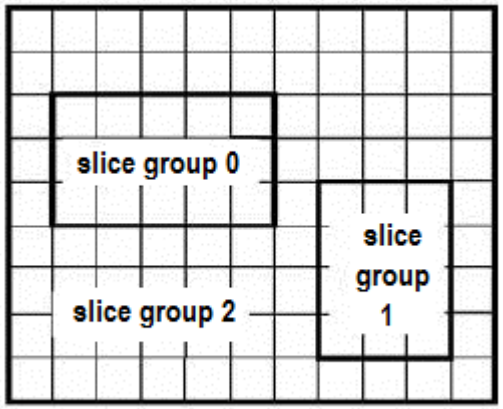
(a) Type 0

(Interleave)



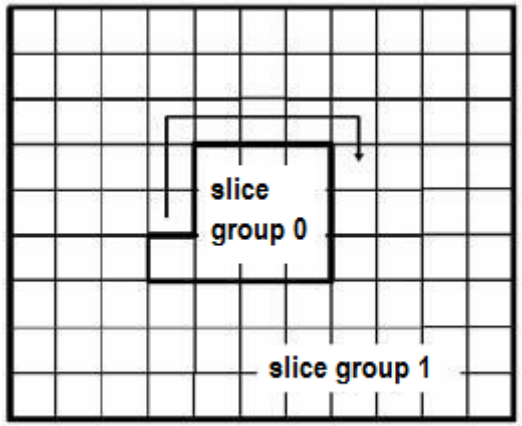
(b) Type 1

(Disperse)



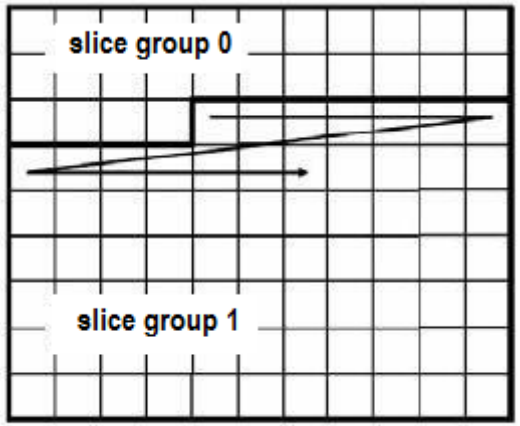
(c) Type 2

(Foreground with left-over)



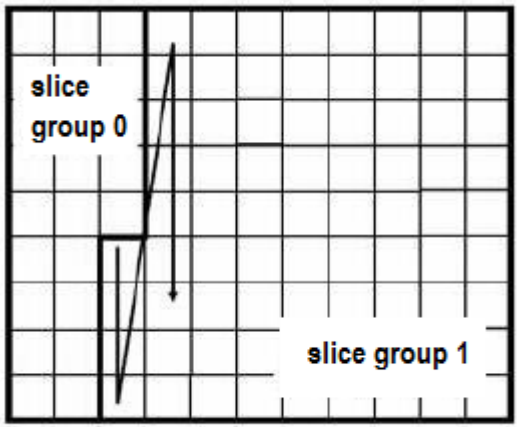
(d) Type 3

(Box-out/Counter box-out)



(e) Type 4

(Raster scan / Reverse)

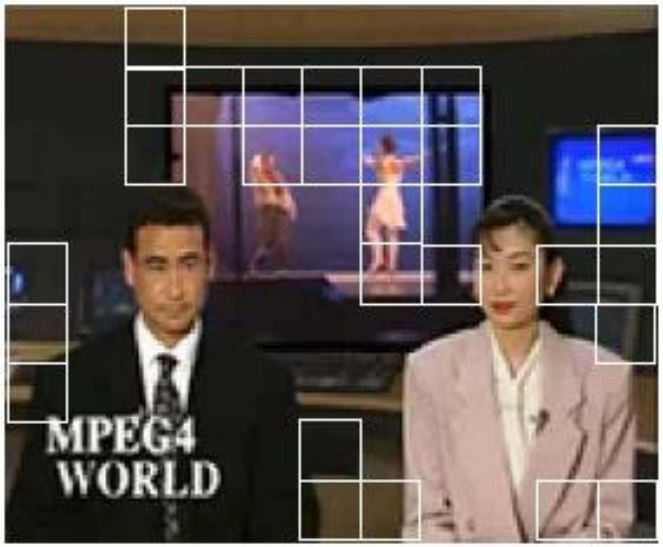


(f) Type 5

(Wipe right / Left)

Inter-Coding: FMO

- Type 6: User defined (Explicit)
  - Need to specify MAmap

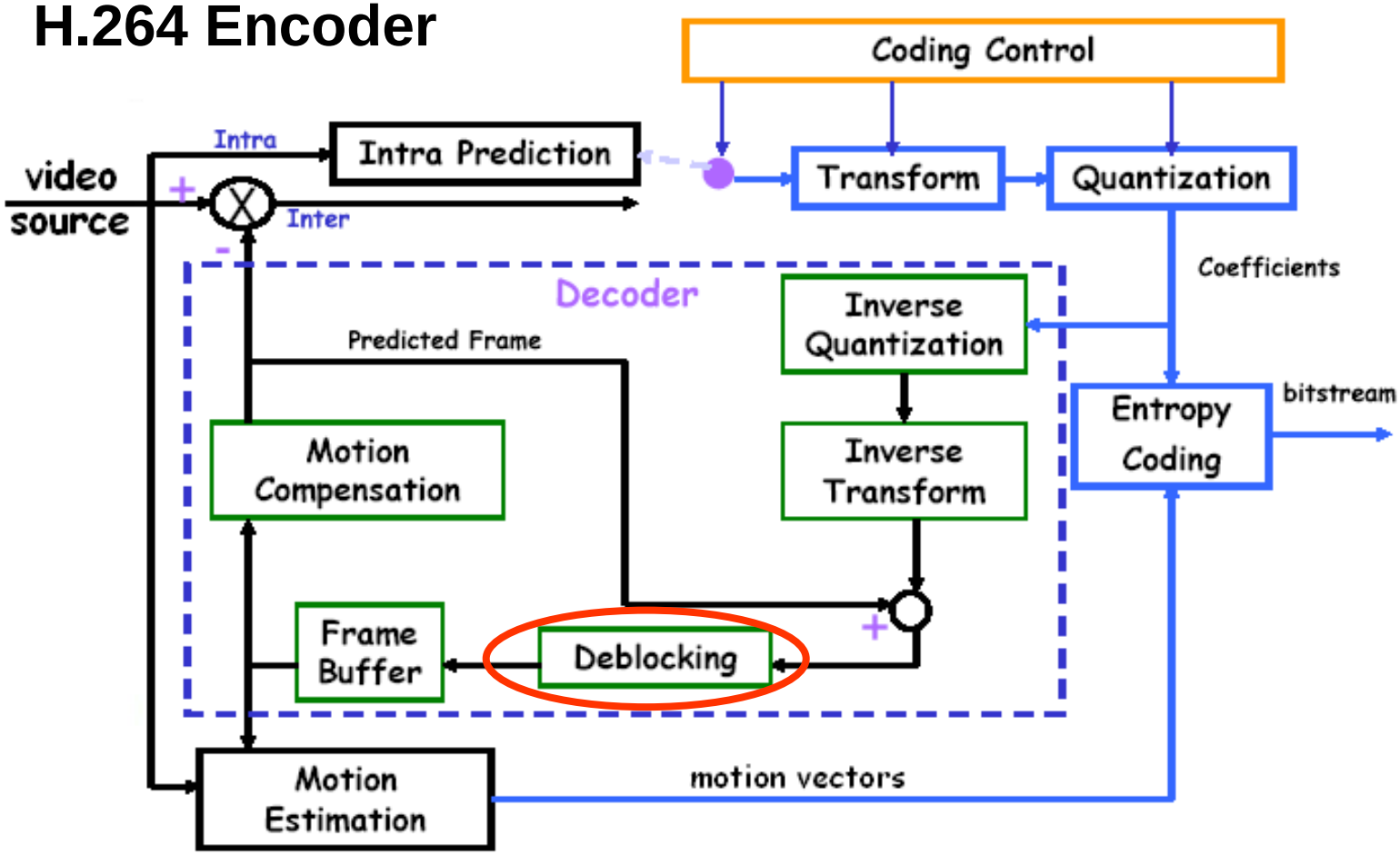


- Original picture with slice-group-1 highlighted
- Slice-group-1 which will cause more distortion when loss is more protected

- Decoded picture with all slice-group-2's MBs being lost
- Temporal copy (co-located) is used for error concealment

Video Coding System

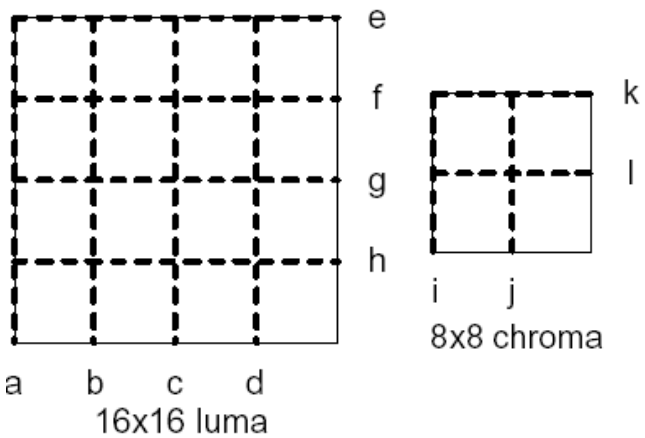
H.264 Encoder



De-blocking filter

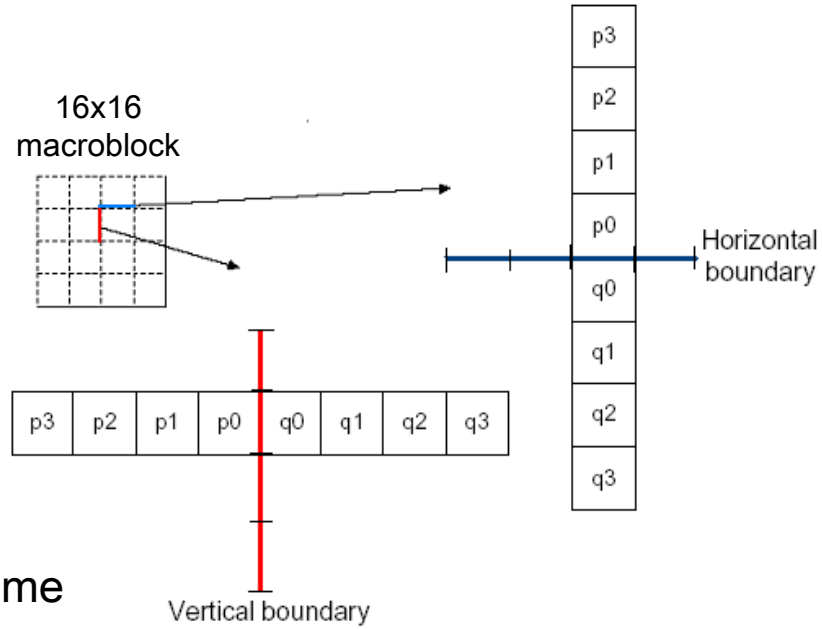
- Purpose of de-blocking
  - improve the appearance of decoded image
  - It is applied to every MB
  - It is applied after inverse transform and before reconstruction for future prediction (Encoder) or display (Decoder).

- Filtering order
  - luma vertical boundaries (a, b, c, d)
  - luma horizontal boundaries (e, f, g, h)
  - chroma vertical boundaries (i, j)
  - chroma horizontal boundaries (k, l)



De-blocking filter

- The filtering can affect up to 3 pixels on each side of boundary.
  - The pixels used for filtering can range from no pixel to five pixels (p0, p1, p2, q0, q1, q2.)
  - ex. 4-tap filter (affect two pixels)
    - p1,p0, q0, q1 → P0, Q0



- Filtering depends on
  - Boundary Strength (Bs)
    - Coding mode: intra > inter
    - MB boundary: yes > no
    - Contain coded coeff: yes > no
    - Ref frame and MV: different > same
    - Use strong filters for large Bs
  - Gradient of Pixels across boundary
    - To "switch off" the filter when there is a significant change across the boundary in original image.

De-blocking filter

Reconstructed, QP=32 (no filter)

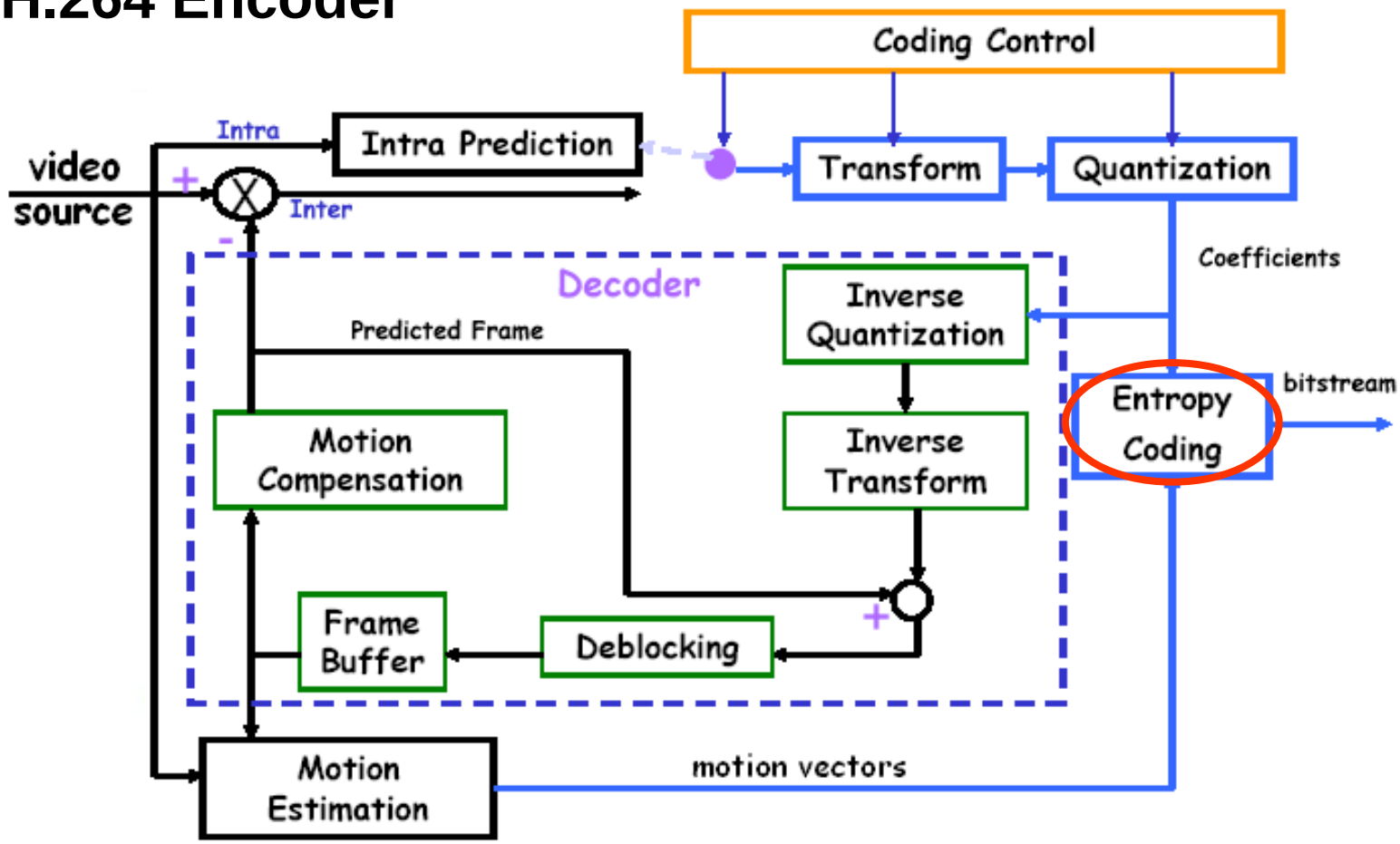
Reconstructed, QP=36 (no filter)

Reconstructed, QP=32 (with filter)

Reconstructed, QP=36 (with filter)

Video Coding System

H.264 Encoder

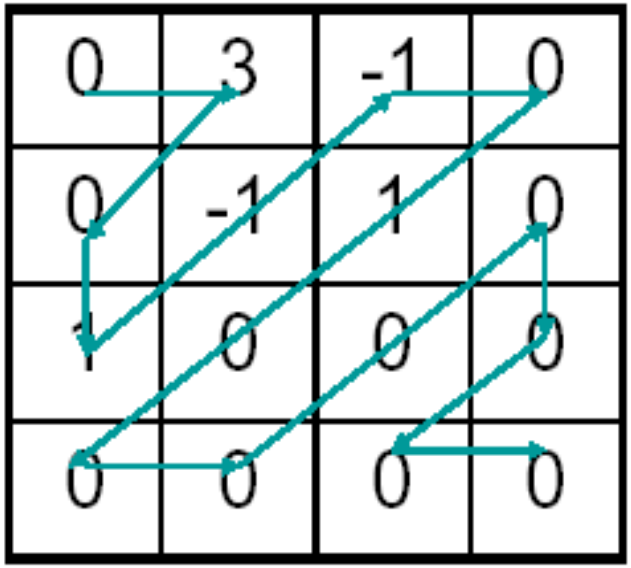


Entropy Coding

- Content-Adaptive Variable Length Coding (CAVLC)
- Content-Adaptive Binary Arithmetic Coding (CABAC)

CAVLC (Content-Adaptive Variable Length Coding)

- Example: residual coding
  - Given reordered block
    - 0,3,0,1,-1,
      -1,0,1,0,0,0,0,0,0,0,0
  - TotalCoeffs = 5
  - T1s = 3
    - T1s: trailing ones.
    - At most 3 trailing ones can be encoded together”



coeff_token

Encoding Example

ZigZag Order →

0,3,0,1,-1,-1,0,1,0,0,0,0,0,0,0,0

← Encoding Order

0	3	-1	0
0	-1	1	0
1	0	0	0
0	0	0	0

Element	Value	Code
coeff_token	TotalCoeffs=5, T1s=3	0000100
T1 sign (4)	+	0
T1 sign (3)	-	1
T1 sign (2)	-	1
Level (1)	+1 (use Level_VLC0)	1
Level (0)	+3 (use Level_VLC1)	0010
TotalZeros	3	111
run_before(4)	ZerosLeft=3; run_before=1	10
run_before(3)	ZerosLeft=2; run_before=0	1
run_before(2)	ZerosLeft=2; run_before=0	1
run_before(1)	ZerosLeft=2; run_before=1	01
run_before(0)	ZerosLeft=1; run_before=1	No code required; last coefficient.

The transmitted bitstream for this block is 000010001110010111101101 .

Entropy Coding - CAVLC

Decoding Example

code: 000010001110010111101101

Code	Element	Value	Output array
0000100	coeff_token	TotalCoeffs=5, T1s=3	Empty
0	T1 sign	+	1
1	T1 sign	-	-1, 1
1	T1 sign	-	-1, -1, 1
1	Level	+1	1, -1, -1, 1
0010	Level	+3	3, 1, -1, -1, 1
111	TotalZeros	3	3, 1, -1, -1, 1
10	run_before	1	3, 1, -1, -1, 0, 1
1	run_before	0	3, 1, -1, -1, 0, 1
1	run_before	0	3, 1, -1, -1, 0, 1
01	run_before	1	3, 0, 1, -1, -1, 0, 1

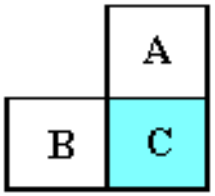
CAVLC (Content-Adaptive Variable Length Coding)

Encode coeff_token

- To encode TotalCoeffs and T1s
- 4 look-up tables: Num_VLC0, Num_VLC1, Num_VLC2, FLC.
- Choose VLC tables according to TotalCoeffs of neighboring blocks.

⇨ **context adaptive**

nC = (nA + nB) / 2



nA, nB: TotalCoeffs of upper and left-hand blocks

Choice of look-up table for coeff_token

nC	Table for coeff_token
0, 1	Num-VLC0
2, 3	Num-VLC1
4, 5, 6, 7	Num-VLC2
8 or above	FLC

Num_VLC0: short code to small TotalCoeffs
Num_VLC1: short code to medium TotalCoeffs
Num_VLC2: short code to large TotalCoeffs
FLC: fixed 6-bits code

⇨ see tables in the next page

VLC table for coeff_token

TrailingOnes (coeff_token)	TotalCoeff (coeff_token)	0 <= nC < 2 Num-VLC0	2 <= nC < 4 Num-VLC1	4 <= nC < 8 Num-VLC2	8 <= nC FLC
0	0	1	11	1111	0000 11
0	1	0001 01	0010 11	0011 11	0000 00
1	1	01	10	1110	0000 01
0	2	0000 0111	0001 11	0010 11	0001 00
1	2	0001 00	0011 1	0111 1	0001 01
2	2	001	011	1101	0001 10
0	3	0000 0011 1	0000 111	0010 00	0010 00
1	3	0000 0110	0010 10	0110 0	0010 01
2	3	0000 101	0010 01	0111 0	0010 10
3	3	0001 1	0101	1100	0010 11
0	4	0000 0001 11	0000 0111	0001 111	0011 00
1	4	0000 0011 0	0001 10	0101 0	0011 01
2	4	0000 0101	0001 01	0101 1	0011 10
3	4	0000 11	0100	1011	0011 11
0	5	0000 0000 111	0000 0100	0001 011	0100 00
1	5	0000 0001 10	0000 110	0100 0	0100 01
2	5	0000 0010 1	0000 101	0100 1	0100 10
3	5	0000 100	0011 0	1010	0100 11
0	6	0000 0000 0111 1	0000 0011 1	0001 001	0101 00
1	6	0000 0000 110	0000 0110	0011 10	0101 01
2	6	0000 0001 01	0000 0101	0011 01	0101 10
3	6	0000 0100	0010 00	1001	0101 11
0	7	0000 0000 0101 1	0000 0001 111	0001 000	0110 00

Entropy Coding

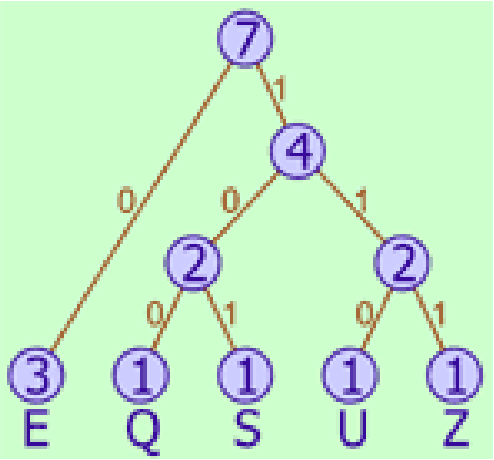
- Content-Adaptive Variable Length Coding (CAVLC)
- Content-Adaptive Binary Arithmetic Coding (CABAC)

Huffman Coding - Review

- **Huffman Coding**
  - build a tree by recursively taking the two least common symbols and turning them into a tree node. Repeat the process until one root node.
  - To compress "SQUEEZE", total bit count: 1x3 + 3x4 = <u>15</u>

Example:

SQUEEZE



Symbol	Bits	Bit count	Entropy
E	0	1	1.22
Q	100	3	2.81
S	101	3	2.81
U	110	3	2.81
Z	111	3	2.81

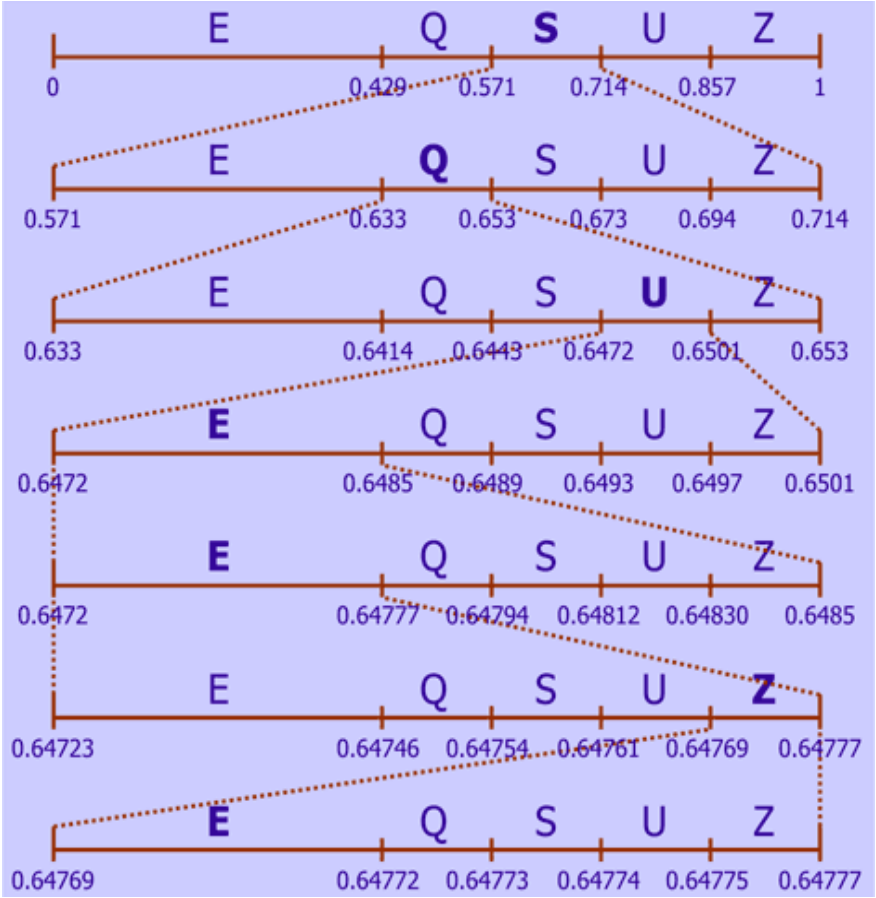
- **Problems with Huffman Coding**
  - cannot encode a symbol in a fractional number of bits

CABAC - Arithmetic Coding

Arithmetic Coding ("SQUEEZE")

- First, range 0-1 is divided into 5 sub ranges, one for each symbol, with length proportional to its probability.
  - E: length= 3/7 (0.429),
  - Each of others: length=1/7 (0.143)
  - S is in the range: 0.571~ 0.714
- Again, subdivide 'S' range into 5 sub ranges in the same way.
  - SQ is in the range: 0.633~0.653
- Repeat the process for other letters.
  - SQUEEZE is in the range: 0.64769~0.64772
- The binary in this range with the smallest number of bits is 0.1010 01011101 (decimal: 0.647705)

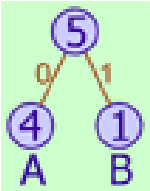
⇨ only 12 bits



Huffman v.s. Arithmetic Coding

- Example 2: Binary symbols ("AABAA")
  - Huffman coding
    - total bit count:

      1x4 + 1x1 = **5 bits**



Symbol	Bits	Bit count	Entropy
A	0	1	0.32
B	1	1	2.32

  - Arithmetic coding
    - AABAA can be represented with a number between 0.512 and 0.594.
    - The binary 0.1001 (0.5625) falls in this range and takes **4 bits** only.

- Huffman Coding
  - Low coding efficiency because it **cannot** encode a symbol in a fractional number of bits. For above example, both A and B use 1 bit even A had a high probability of 80% and B a low probability of 20%.
- Arithmetic Coding
  - An entire word (or message) is coded as a single number in the range of 0 to 1. For longer messages and skewed probabilities, the coding gains are even greater.

Entropy Coding - CABAC

- CABAC **Context Model**
  - There are a many **context models** defined in H.264 standard for the various syntax elements.
    - to estimate the conditional probabilities of symbols (0 / 1)

0	0	0	0
0	0	0	0
0	0	1	1
0	0	1	1

  - Bit-plane coding
    - Bin n of a syntax of all mb in a frame form a bit-plane.
      - Assume a bit-plane of mb_type of a frame with 16 mb.
    - Different bit-planes may use different context model.
    - Some bit-planes maybe better for left-up context model, some maybe better for the previous bit context model.

Entropy Coding - CABAC

SYNTAX ELEMENTS AND ASSOCIATED RANGE OF CONTEXT INDICES

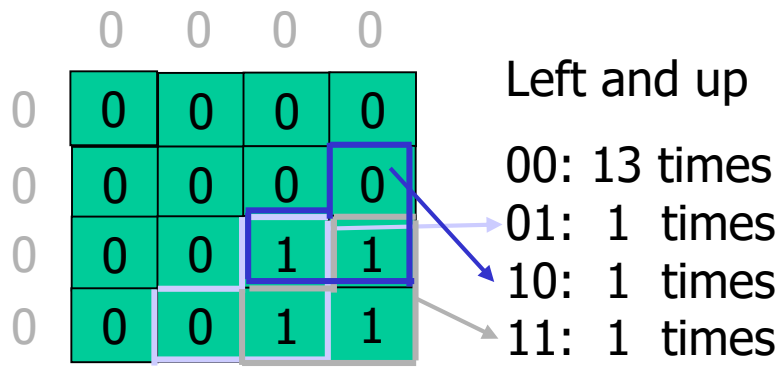
Syntax element	Slice type		
	SI/I	P/SP	B
mb_type	0/3-10	14-20	27-35
mb_skip_flag		11-13	24-26
sub_mb_type		21-23	36-39
mvd (horizontal)		40-46	40-46
mvd (vertical)		47-53	47-53
ref_idx		54-59	54-59
mb_qp_delta	60-63	60-63	60-63
intra_chroma_pred_mode	64-67	64-67	64-67
prev_intra4x4_pred_mode_flag	68	68	68
rem_intra4x4_pred_mode	69	69	69
mb_field_decoding_flag	70-72	70-72	70-72
coded_block_pattern	73-84	73-84	73-84
coded_block_flag	85-104	85-104	85-104
significant_coeff_flag	105-165, 277-337	105-165, 277-337	105-165, 277-337
last_significant_coeff_flag	166-226, 338-398	166-226, 338-398	166-226, 338-398
coeff_abs_level_minus1	227-275	227-275	227-275
end_of_slice_flag	276	276	276

Entropy Coding - CABAC

- CABAC **Context Model**
  - Given a bit-plane of mb_type with 16 mb.

0	0	0	0
0	0	0	0
0	0	1	1
0	0	1	1

  - Without context model,
    - For symbol 0 and 1, P(0)=12/16, P(1)=4/16.
    - Entropy: 12/16log2(16/12)+4/16log2(16/4) = 0.81
  - Assume left-up context model



P(0|00) = 12/13
P(1|00) = 1/13
P(0|01) = 0
P(1|01) = 1
P(0|10) = 0
P(1|10) = 1
P(0|11) = 0
P(1|11) = 1

0 bit

- Entropy: $12/16\log_2(13/12)+1/16\log_2(13/1)+3/16\text{x}0 = 0.30875$

→ less than 0.81

Entropy Coding - CABAC

- **CABAC** (Context-Adaptive Binary Arithmetic Coding)
  - *Context-adaptive* modeling
    - estimate the conditional probabilities of symbols
    - Switch between several models according to prior symbols
  - *Arithmetic* codes
    - Non-integer number of bits per symbol
    - Beneficial for skewed symbol probability distribution.
  - *Adaptive* arithmetic codes
    - Adapt to non-stationary symbol statistics.
  - Adaptive *binary* arithmetic coding
    - Binarization is needed for non-binary symbols
    - Simple and fast
  - Average bit-rate saving
    - over CAVLC 10-15%

Chapter 1: Source Coding- Video

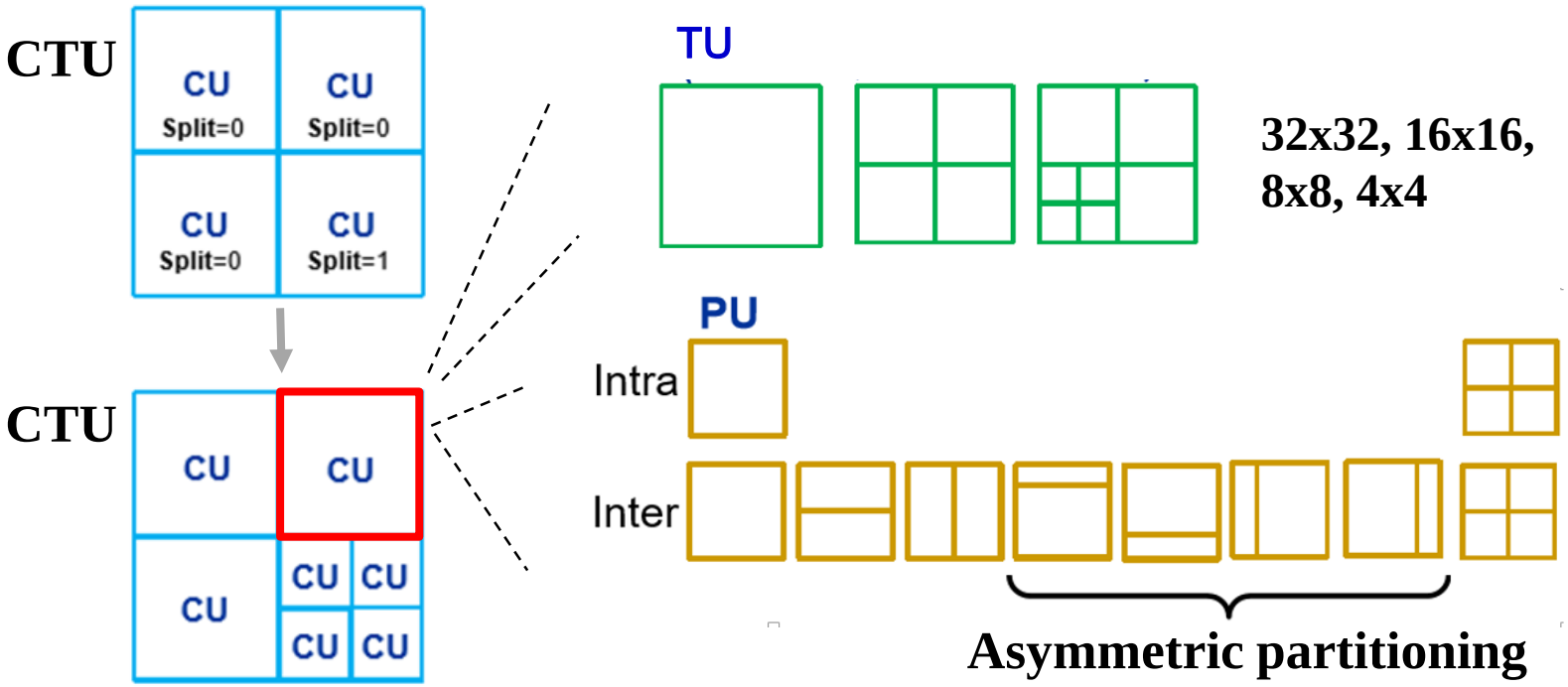
- **Video Coding – MPEG2**
- **Advanced Video Coding (H.264/AVC)**
- **High Efficient Video Coding (H.265HEVC)**

Large Coding Blocks: CTU

- Each frame is broken up into blocks
- Large block sizes reduce signaling overhead
- In H.264, **macroblock** is always **16x16** pixels
  - Each macroblock is *either* **inter** or **intra** coded
- In HEVC, **Coding Tree Unit (CTU)** can have up to **64x64** pixels
  - CTU can have a *combination* of **inter** and **intra** coded blocks
  - A CTU consists of an NxN luma block and two chroma blocks
    - N =16, 32, or 64
    - Large CTBs are particularly beneficial for high-resolution video content.
  - A CTU can be subdivided into Coding Units (**CU**), Prediction Units (**PU**), and Transform Units (**TU**).

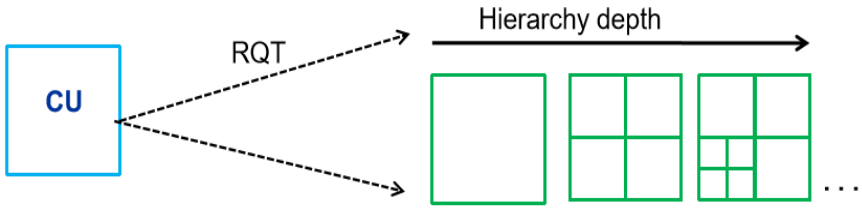
Flexible Coding Structure: CU

- **CTU**: divided into square Coding units (**CU**) with quad-tree structure using ***split*** flag
- **CU**: divided into Prediction units (**PU**) and **TU** (Transform unit)
  - PUs in a CU share the same mode: - intra, inter or skipped



Transform

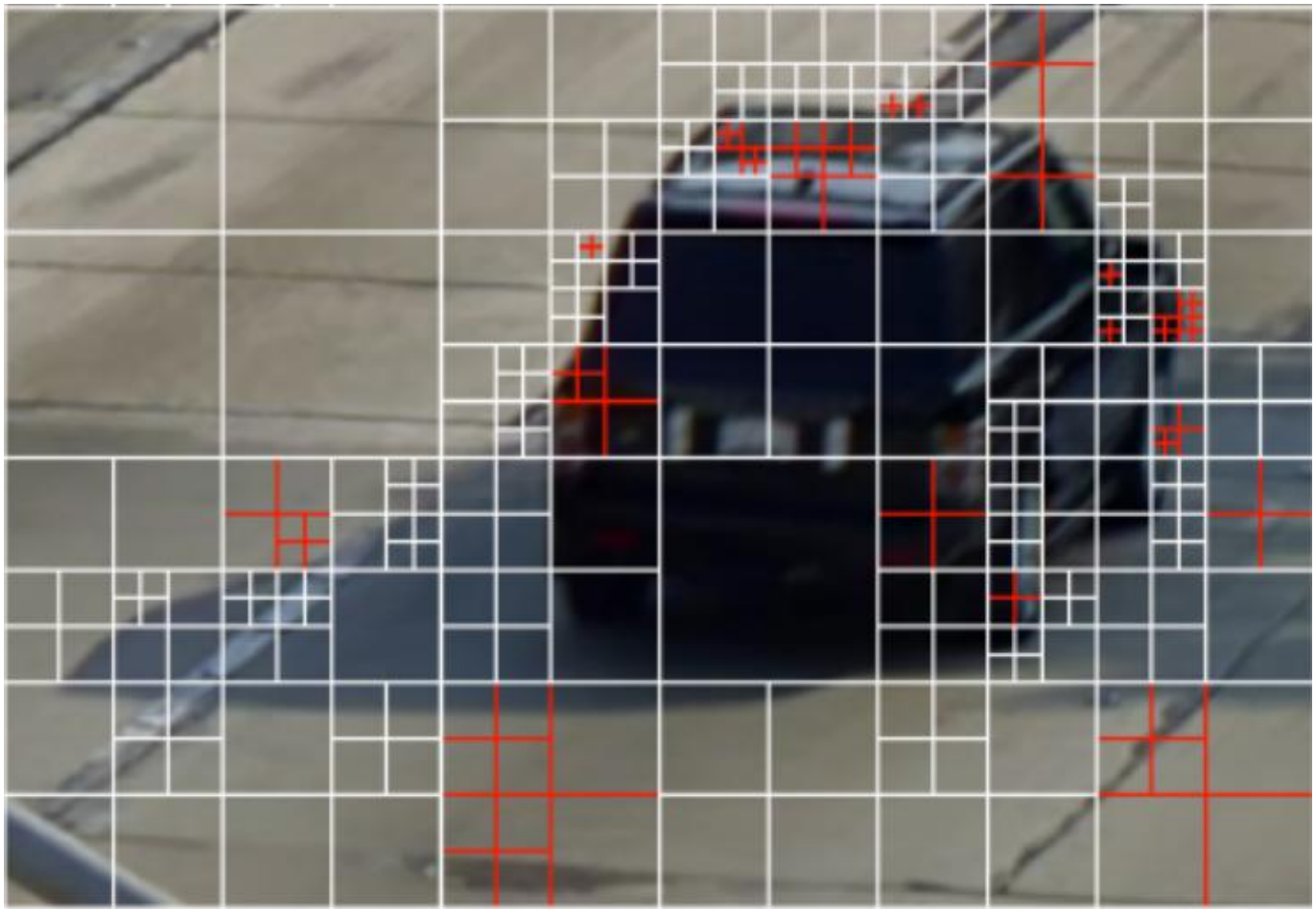
- **H.264:** 4x4 Integer transform for all block partitions and modes
  - Adaptive 8X8 transform in High Profile.
- **HEVC:** supports 4x4, 8x8, 16x16, 32x32 integer transforms
  - Two types of transforms: **IDCT** and **IDST**(only for Intra 4x4 luma)
- Transform Unit (TU) size is specified for each CU
  - TU cannot across intra PU boundary



- Increase 5 to 10% coding efficiency
- Increased complexity compared to H.264/AVC
  - 8x more computations per coefficient, 16x larger transpose memory

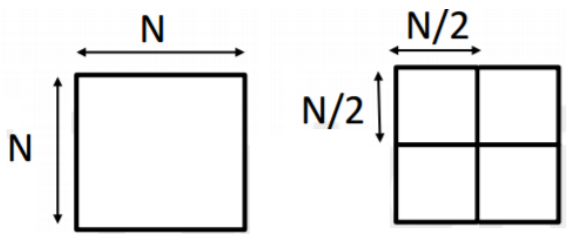
Example

CU (white) and nested TU (red) partitioning in a 4k × 2k sequence



Prediction Unit (PU)

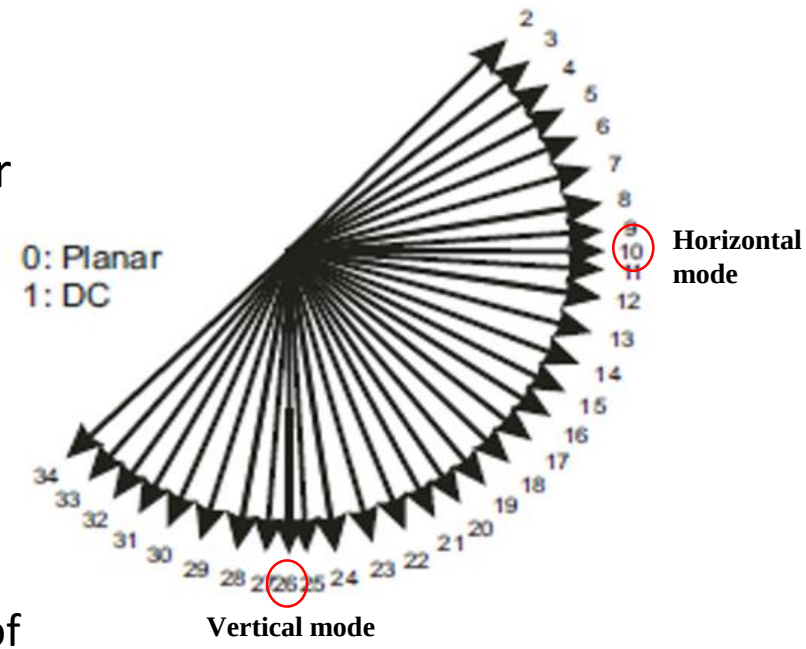
- **Intra-Coded CU** can only be divided into square prediction units (PUs)
  - **64x64, 32x32, 16x16, 8x8, 4x4**

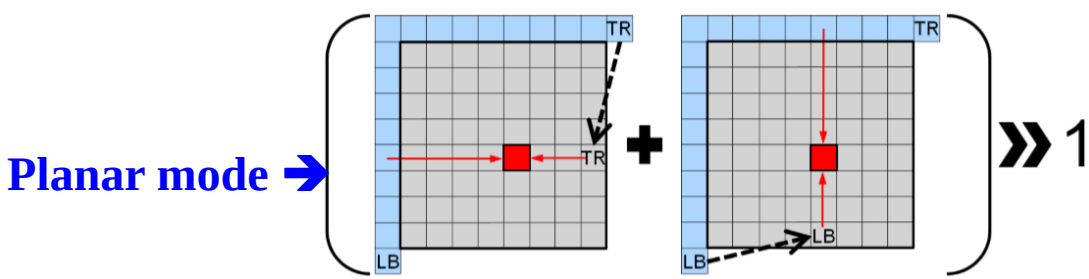


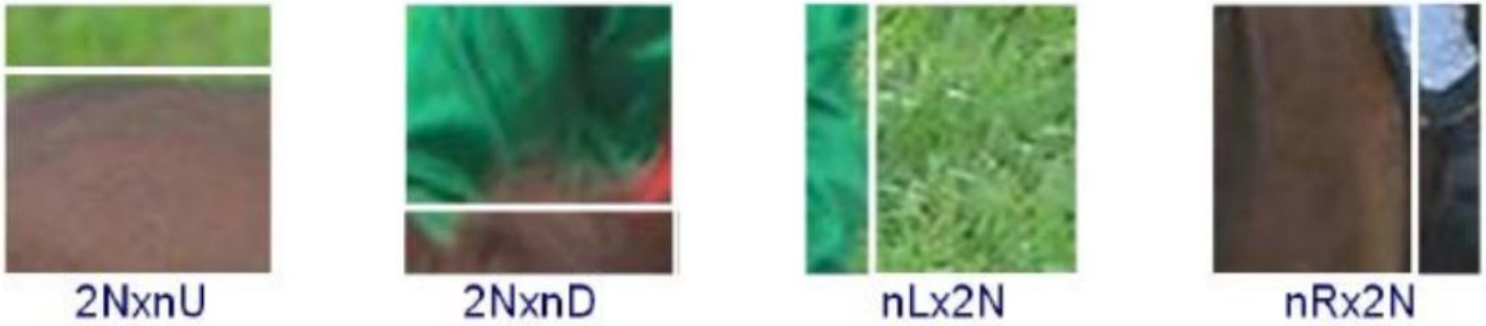
- **Inter-Coded** CU can be divide into square and non-square PU as long as one side is at least 4 pixels wide (no 4x4 PU)
  - **64x64, …, 8x8 (no 4x4)**

Prediction Unit (PU)

- **Intra prediction**
  - H.264/AVC has 10 modes
    - 4X4: angular (8 modes), DC, planar
    - 16X16: DC, planar, H, V
  - HEVC has 35 modes
    - angular (33 modes), DC, planar
    - PB sizes from 4×4 up to 64×64.
    - 3 most probable modes (MPM)
  - DC – Constant value which is an average of neighboring pixels (ref. samples)
  - Planar – Average of horizontal and vertical prediction





Asymmetric Motion Partition (AMP)

- H.264
  - 16x16, 16x8, 8x16 or 8x8 partitions
  - 8x4, 4x8 or 4x4 sub-partitions for 8x8 partitions
- HEVC
  - More flexible, symmetric and asymmetric PB partitioning